Project ID : 17

A
Project Report On

Hotel Five

Live and Baar — A Web-Based Hospitality and Food Ordering System

Submitted in the Partial fulfillment of the requirement for awarding the degree of Bachelor of Computer Application (BCA)
SEM 5, Year 2026-27

By
Kapadiya Foram
(Enrollment No. 240431108)

Under the Guidance of
Prof. Kashul Jani



Faculty of Computer Application,
Noble BCA College
Noble UNIVERSITY – JUNAGADH



Noble University, Junagadh

Certificate

Enrollment No. 240431108

This is to certify that the Project / practical work was satisfactorily carried out and hence submitted this report is the bonafide work of Mr./ Ms. Kapadiya Foram

Student of Bachelor of Computer Applications Sem 5 in the Laboratory of Noble University, Junagadh during the academic year 2026-27.

Prof. Kashul Jani **Prof. Bhargav Rajgor**

Internal Guide **Principal**

Preface

This report contains all the outputs of a project undertaken to develop a working model of **Hotel Five — Live and Baar**, a web-based hospitality and food ordering system, and to study the existing working system. As part of this study, we analyzed the requirements of the working model and tried to improve it using the technological functionality that the current computer science provides.

This volume contains the documentation summary of the working system, the background study and analysis, the technical requirements and the working specification of the developed system.

We recommend the user to take the advantage of the study presented here and implement the system with their required improvisation. We hope this study will reduce the burden of the reanalysis of the development phase.

Acknowledgment

“It is not possible to prepare a project report without the assistance & encouragement of other people. This one is certainly no exception.”

On the very outset of this report, We would like to extend our sincere & heartfelt obligation towards all the personages who have helped us in this endeavor. Without their active guidance, help, cooperation & encouragement, We would not have made headway in the project. We are ineffably indebted to our HOD **Mr. Paresh Vora** Sir for his conscientious guidance and encouragement to accomplish this assignment.

We are extremely thankful and pay our gratitude to our project guide Prof **Kashul Jani** for his valuable guidance and support on completion of this project in its presently.

We extend our gratitude to Faculty of Computer Application, Noble University, Junagadh for giving us this opportunity.

Thanking You

Kapadiya Foram (240431108)

INDEX

Sr. No.	Title		Page No.
1	**University Certificate**		i
2	**Preface**		ii
3	**Acknowledgement**		iii
4	**Index**		iv
5	**Project Profile**		**1**
	1.1	Definition	1
	1.2	Scope	1
	1.3	Objective	2
6	**System Analysis and Specification**		**4**
	2.1	Existing System	4
	2.2	Limitations of Existing System	4
	2.3	Feasibility Study	5
	2.4	Need of New System	6
	2.5	Front End & Back End Tools	7
7	**System Requirement Specification**		**9**
	3.1	Proposed System and Advantages	9
	3.2	Development Strategy Model	12
8	**System Design**		**14**
	4.1	Data Flow Diagram	14
	4.2	E-R Diagram	16
	4.3	Class Diagram	17
	4.4	Use Case Diagram	18
	4.5	Sequence Diagram	20
	4.6	Activity Diagram	22
	4.7	Data Dictionary (Database)	23
9	**Screen Design**		**26**
	5.1	Input Design	26
	5.1.1	Home Page	26
	5.1.2	Services Section	27
	5.1.3	About Us Page	28
	5.1.4	Statistics and Photo Strip	29
	5.1.5	Our Rooms Page	30
	5.1.6	Our Menu Page	30
	5.1.7	Gallery Page	31
	5.1.8	Popular Hotels Page	32
	5.1.9	Team Page	32
	5.1.10	Reviews Page	33
	5.1.11	Order Food Page	34
	5.1.12	Login Page	34
	5.1.13	Sign Up Page	35
	5.1.14	Payment Page	36
	5.2	Output Design / Report Design	38
10	**Coding (Few Code Snippets)**		**40**
11	**Testing (Manual, Test Cases and Test Data)**		**46**
	7.1	Manual Testing	46
	7.2	Test Cases	47
	7.3	Test Data	48
12	**Enhancements**		**50**
	8.1	Advantages of Your Project	50
	8.2	Limitation of Your Project	51
	8.3	Future Scope	51
13	**References**		**54**

1- Project Profile

1.1 Definition

This project, titled **“Hotel Five”**, is a web-based hospitality and food-ordering platform developed to bring the everyday activities of a hotel and its in-house restaurant onto a single digital interface. The system allows a visitor to explore the property, study the available rooms and suites, browse the restaurant menu, register an account, log in securely, place a food order, complete a payment entry, and submit a review of the service received. Instead of depending on telephone calls, handwritten registers and informal messages, the project converts these interactions into structured digital forms whose results are stored permanently in a relational database.

The platform is composed of a set of presentation pages and a set of transactional modules. The presentation pages include the Home page, the About page, the Services section, the Our Rooms page, the individual room detail pages, the Gallery, the Popular Hotels page and the Team page. These pages are responsible for communicating the identity of the brand, explaining what the property offers and building the confidence of a first-time visitor.

From a technical point of view, the project follows a classic three-layer web architecture. The presentation layer is written in HTML5, CSS3 and JavaScript. HTML provides the structure of every page, CSS controls the colour scheme, spacing, card layouts, hero banners and responsive behaviour, and JavaScript supplies interactivity such as scroll-reveal animations, animated counters, quantity controls and client-side validation of every form.

Communication between the browser and the server is handled with the native **Fetch API**. When a user submits a form, JavaScript collects the field values into a *FormData* object and posts them asynchronously to the corresponding PHP endpoint. The endpoint answers with a JSON document containing a status flag and a human-readable message.

Security has been considered at the level appropriate to an academic prototype. User passwords are never stored in readable form; they are passed through the PHP *password_hash()* function using the BCRYPT algorithm before insertion, and verified later with *password_verify()*. Payment card numbers are never persisted in full.

In simple words, the project is a complete front-of-house website for a hotel and restaurant. It presents the property attractively, it lets guests act on what they see, and it records those actions in a database so that nothing depends on memory or paper. It is useful both as a demonstration of full-stack web development using HTML, CSS, JavaScript, PHP and MySQL, and as a realistic starting point for a small hospitality business that wishes to move its operations online.

1.2 Scope

The scope of the “Hotel Five” project defines the boundary of what the present version of the system is expected to do, which categories of users it serves, which data it is responsible for, and which areas are deliberately left for future development. Stating the scope precisely is important because it prevents the project from being judged against expectations it was never designed to meet, and it gives a clear starting point for anyone who continues the work later.

Functional Scope

The current implementation supports the following operations end to end: browsing every informational page of the site; viewing the catalogue of rooms together with their occupancy, area, floor, view and amenity details; browsing the restaurant menu with descriptions, ratings and

prices; creating a new user account through the sign-up form; logging in with an existing account; placing a food order by selecting an item and a quantity; submitting billing and card details on the payment page; and submitting a star rating with a written review. Every one of these actions that produces data results in a row being written to the MySQL database.

Technical Scope

The application is built without any external framework. The front end consists of hand-written HTML pages supported by custom CSS files and vanilla JavaScript; the only third-party front-end resource is the Font Awesome icon library, loaded from a CDN.

User Scope

Two categories of user are served by the present version. The first is the **visitor**, an anonymous guest who can browse every public page, view rooms and food, read reviews and create an account.

Out of Scope

The following items are explicitly outside the boundary of the current project: a live payment gateway and any real financial settlement; an administrative dashboard with record management; a complete room-booking workflow with availability checking; role-based authorisation and protected routes; automated e-mail or SMS notifications; reporting, charting and business analytics; automated unit or integration test suites; and deployment to a public production server. Each of these is discussed in Chapter 8 as part of the future scope.

1.3 Objective

The objectives of the project describe what the system is intended to achieve, both for the end user and for the developer. They were fixed at the requirement stage and were used throughout development as the criteria against which each module was judged.

1. **To build a professional hospitality website.** The first objective is to produce a site whose visual quality is comparable to a commercial hotel website, because in the hospitality sector the presentation itself is part of the product being sold.
2. **To present hotel services in a structured manner.** The Services section groups the offerings of the property — luxury rooms, fine dining, spa and wellness, swimming pool, lounge and bar, and in-room delivery — into a readable grid so that a visitor understands the full range of facilities within a few seconds.
3. **To publish the room catalogue with complete information.** Each room card carries the room name, view and floor, guest capacity, area in square metres, connectivity, amenity icons, the nightly tariff in rupees and a direct booking action.
4. **To allow account creation through a validated sign-up form.** The registration module must collect the full name, e-mail address, telephone number and password, and must reject incomplete, malformed or duplicate submissions before any data reaches the database.
5. **To allow returning users to authenticate securely.** The login module must verify credentials against stored records and must give distinct, accurate feedback for an unknown account and for an incorrect password.
6. **To digitise the food-ordering process.** A customer must be able to select a menu item and a quantity, supply delivery details and submit an order that is priced by the server and stored as a database record.
7. **To provide a checkout and billing step.** The payment page must collect billing identity, address and card details, compute or receive the payable amount, and record the transaction

while storing only a masked reference to the card.

8. **To capture guest testimonials systematically.** The review module must collect a name, an e-mail address, a rating between one and five, and a review text of meaningful length, and must store it centrally rather than leaving feedback scattered across informal channels.
9. **To demonstrate a modern asynchronous browser-to-server workflow.** Every form submission must travel through the Fetch API and return JSON, so that the interface updates without a full page reload.
10. **To improve customer engagement through interface quality.** Scroll-reveal animations, animated counters, hover effects on cards and a consistent gold-on-navy colour identity are used to hold the attention of the visitor.
11. **To satisfy the academic requirement of a full-stack project.** The work must show competence in interface design, form handling, client-side and server-side validation, database schema design, database connectivity, session handling and basic application security.

2- System Analysis and Specification

2.1 Existing System

The existing system, in the context of this project, refers to the way a small or medium hotel with an attached restaurant typically manages its guest-facing activities before any dedicated software is introduced. Understanding that situation in detail is necessary, because the value of the proposed system lies precisely in the inefficiencies it removes.

In the conventional arrangement, a prospective guest first learns about the property through word of mouth, a listing on a third-party travel portal, a social media page, or a printed brochure. None of these channels presents the complete picture. A travel portal may show two photographs and a tariff but nothing about the restaurant; a social media page may show food photographs but no room information; a brochure becomes outdated as soon as a tariff changes.

When the guest decides to act, the interaction moves to the telephone. Room enquiries, tariff questions, availability checks and food orders are all handled by a staff member who answers the call, writes the details on a pad or in a register, and repeats them back for confirmation. The same staff member is frequently also attending to guests at the counter.

Food ordering in particular suffers under this arrangement. The customer cannot see the menu while speaking, so the staff member has to read out items and prices. Quantities and special instructions are transcribed by hand.

Payment follows the same informal pattern. Cash is received and noted, or a card is processed on a terminal whose receipt is filed separately from the order slip. Reconciling the day’s orders against the day’s receipts becomes a manual exercise at closing time, and any mismatch is difficult to trace back to a specific transaction.

Guest feedback, which is the most valuable information a hospitality business can collect, is the least systematically handled of all. Comments are made verbally at checkout and forgotten. Written remarks go into a visitors’ book that nobody analyses.

Some properties take a partial step towards digitisation by creating a static informational website. This helps with presentation but not with operations: the site displays photographs and a contact number, and every actual transaction still returns to the telephone. Others rely on messaging applications, which produce a searchable but completely unstructured record spread across dozens of individual chats.

The problems that the proposed system is intended to address can therefore be summarised as follows: information about the property is fragmented across channels; every transaction depends on a human intermediary; records are unstructured, inconsistent and easily lost; there is no reliable link between an order and its payment; and guest feedback is collected, if at all, in a form that cannot be analysed.

2.2 Limitations of Existing System

The following limitations of the conventional arrangement were identified during the analysis phase and directly shaped the requirements of the new system.

1. **Fragmented information.** Room details, tariffs, restaurant menu, facilities and photographs live in different places and are rarely updated together. A guest cannot form a complete picture of the property from any single source, and the business cannot guarantee that the information a guest is acting upon is current.
2. **Dependence on manual communication.** Nearly every transaction requires a telephone call

or a personal visit, which means it can only happen during working hours and only when a staff member is free. Guests in different time zones, guests who prefer not to call, and guests browsing late at night are all effectively excluded.

3. **Unstructured record keeping.** Orders on paper slips, payments on terminal receipts and customer details in a register cannot be searched, sorted, counted or cross-referenced. Producing even a simple statement such as “how many pizzas were sold last week” requires manually reading through every slip.
4. **High error rate.** Every transcription from speech to paper introduces the possibility of a wrong quantity, a misheard address, a wrong telephone digit or an arithmetic mistake in the total. Because there is no authoritative record, such errors are discovered only when the guest complains.
5. **No customer identity.** Without accounts, the business cannot recognise a returning guest. Every interaction starts from zero, the guest repeats the same details each time, and no history of preferences or previous orders is available to the staff.
6. **Weak data protection.** Personal details written in an open register are visible to anyone at the counter. Card details noted on paper are a serious liability.
7. **Feedback is not captured.** Because there is no structured channel for reviews, the business loses both the operational insight that complaints would provide and the marketing value that positive testimonials would provide.
8. **Poor scalability.** The manual arrangement works acceptably at low volume, but the staff time required grows in direct proportion to the number of transactions. At peak season, the same process that handled ten calls a day comfortably becomes the bottleneck of the entire operation.
9. **No measurement.** Because nothing is recorded in a structured form, management decisions about menu composition, tariff revision, staffing levels and marketing spend are made on impression rather than on evidence.

2.3 Feasibility Study

A feasibility study was carried out before development began, in order to confirm that the proposed system could actually be built and operated with the resources available. Five dimensions were examined.

Technical Feasibility. The project is technically feasible because every component of the proposed solution is a mature, freely available and well-documented technology. HTML5, CSS3 and JavaScript are supported natively by every modern browser and require no installation. PHP provides the server-side scripting capability, and the MySQLi extension bundled with it provides database connectivity without any additional driver.

Economic Feasibility. The project is economically feasible because its direct cost is effectively zero. Every tool in the stack is open-source or free of charge: XAMPP, PHP, MySQL, the browser and the code editor. The only third-party resource, the Font Awesome icon library, is used through its free CDN distribution.

Operational Feasibility. The project is operationally feasible because the workflows it implements mirror the actions a guest already performs mentally. Browsing rooms is equivalent to looking through a brochure; ordering food is equivalent to reading a menu and naming an item and a quantity; leaving a review is equivalent to telling the manager what one thought of the stay. Because the digital version of each action follows the familiar order, a first-time visitor needs no instruction and no training.

Schedule Feasibility. The project is feasible within an academic semester. The architecture is deliberately simple, with a limited number of pages, five database tables and five back-end endpoints. Because the modules are independent of one another, they could be built and tested in sequence: the static pages first, then the database schema, then registration and login, then ordering, then payment, and finally reviews.

Security and Legal Feasibility. At the level of an academic prototype the security requirements are satisfied. Passwords are hashed with BCRYPT, card numbers are truncated to their last four digits before storage, all input is validated on the server as well as in the browser, and the login query uses a prepared statement with bound parameters. It must be stated honestly, however, that the system is not compliant with the standards that a real payment application would have to meet. There is no PCI-DSS compliance, no TLS configuration, no CSRF protection, no rate limiting on the login endpoint and no audit logging. For this reason the payment module is presented as a simulated checkout that records billing data, and not as a live financial transaction facility. Within that clearly stated boundary the project is legally and ethically sound.

2.4 Need of New System

The need for the new system follows directly from the limitations catalogued in Section 2.2. Each requirement below is stated together with the specific deficiency it removes.

A single authoritative source of information. Rooms, facilities, menu, photographs and tariffs must be published in one place that the business controls and can update in one operation. This removes the fragmentation described in Limitation 1 and guarantees that every guest sees the same current information.

Round-the-clock self-service. Guests must be able to browse, register, order and review at any hour without a staff member being present. This removes the dependence on manual communication described in Limitation 2 and extends the effective service window from office hours to the full day.

Structured, queryable records. Every order, payment and review must be written into a typed database column with a timestamp, so that the history of the business can be searched, counted and reconciled. This removes the unstructured record keeping of Limitation 3 and the absence of measurement described in Limitation 9.

Validated data capture. Because the guest types their own name, telephone number and address directly into a validated form, the transcription step disappears entirely. Format rules are enforced before the data is accepted, which addresses the high error rate of Limitation 4.

Persistent customer identity. A registration and login facility gives each guest a stable identity in the system, which addresses Limitation 5 and lays the foundation for order history, saved preferences and personalised offers in later versions.

Protection of sensitive data. Hashed passwords, masked card references and server-side sanitisation replace the open register and the handwritten card note of Limitation 6.

A structured feedback channel. A dedicated review form with a numeric rating and a minimum text length converts the informal comments of Limitation 7 into analysable records that can also be republished as testimonials.

Capacity to absorb growth. Because a form submission consumes no staff time, the system handles the hundredth order of the day exactly as quickly as the first, which addresses the scalability ceiling described in Limitation 8.

Improved presentation and competitiveness. In a market where guests compare properties

visually before they compare prices, a modern, animated, responsive website is itself a commercial necessity rather than a decoration.

2.5 Front End & Back End Tools

This section lists every technology used in the project, its purpose, the part of the system in which it is applied and any relevant version information.

Front End Tools

Technology	Purpose	Where Used	Notes / Version
HTML5	Structure and semantic content of every page	index.html, about.html, room.html, room-1 to room-8.html, order.html, payment.html, review.html, login.html, sign-up.html, galary.html, popular.html, our_food.html, burger.html, team.html	Living standard; no external library
CSS3	Colour identity, typography, card and grid layout, hero banners, hover effects, animations and responsive breakpoints	css/ directory, linked by every page	Custom stylesheets, hand written
JavaScript (ES6)	Client-side validation, scroll-reveal animation, animated counters, quantity controls, navigation behaviour, form submission and response handling	Inline and linked scripts on all pages	Native browser API, no framework
Fetch API	Asynchronous transport of form data from the browser to PHP endpoints and reception of JSON responses	sign-up.html, login.html, order.html, payment.html, review.html	Browser native
FormData API	Collection of form field values into a transmittable object	All submitting pages	Browser native
localStorage	Retention of the logged-in display name and the pending order total between pages	login.html, order.html, payment.html	Browser native
Font Awesome	Vector icons for services, amenities, ratings, navigation and social links	All pages, via CDN stylesheet	Version 6.5.0 declared in markup

Back End Tools

Technology	Purpose	Where Used	Notes / Version
PHP	Server-side processing: sanitisation, validation, business rules, database interaction, session creation and JSON response generation	php/db_connect.php, register.php, login.php, order.php, payment.php, review.php	XAMPP-bundled runtime
MySQLi extension	Connection to MySQL, prepared statements, parameter binding, result retrieval and character-set configuration	All back-end scripts through db_connect.php	Native PHP extension
MySQL	Persistent relational storage of users, orders, payments, reviews and bookings	Database *food_website*	XAMPP-bundled server
SQL	Schema definition and data manipulation	php/database/ setup scripts and php/luxe_hotel.sql	MySQL dialect
PHP Sessions	Retention of authenticated user identity on the server across requests	php/login.php	Native session API
password_hash / password_verify	One-way hashing of user passwords and verification at login	register.php, login.php	BCRYPT algorithm

Justification of the Technology Choice

The stack was chosen deliberately rather than by default. A framework such as Laravel or React would have provided structure and convenience, but it would also have hidden the underlying mechanics that this project is intended to demonstrate. By writing the routing, the validation, the query construction and the session handling directly, the work shows a clear understanding of what a framework does on the developer's behalf.

3- System Requirement Specification

3.1 Proposed System and Advantages

The proposed system is a lightweight but complete digital hospitality platform built on HTML, CSS, JavaScript, PHP and MySQL. It replaces the fragmented, telephone-driven arrangement described in Chapter 2 with a single website through which a guest can inform themselves, identify themselves, transact and give feedback, while every action leaves a structured record in a relational database.

Overall Architecture

The system follows a three-tier architecture. In the **presentation tier**, the browser renders HTML pages styled with CSS, and JavaScript supplies interactivity and first-line validation. In the **application tier**, PHP scripts receive posted data, sanitise and validate it, apply business rules such as price calculation and card masking, and emit a JSON response.

The runtime sequence for any transactional action is identical throughout the system. The user fills a form and presses the action button. JavaScript checks the field values and, if anything is wrong, halts the submission and shows an inline message without contacting the server.

Module Composition

Module	Responsibility	Files Involved
Presentation	Brand communication, service overview, room catalogue, gallery, team, popular properties	index.html, about.html, room.html, room-1 to room-8.html, galary.html, popular.html, team.html
Registration	Account creation with duplicate-email prevention and password hashing	sign-up.html, php/register.php
Authentication	Credential verification and session establishment	login.html, php/login.php
Ordering	Menu presentation, item and quantity selection, server-side pricing	our_food.html, burger.html, order.html, php/order.php
Payment	Billing capture, card masking, transaction recording	payment.html, php/payment.php
Review	Rating and testimonial capture with quality constraints	review.html, php/review.php
Data Access	Central connection configuration and character-set handling	php/db_connect.php
Installation	Creation of the database and all tables on a fresh machine	php/database/*.php, php/luxe_hotel.sql

Advantages of the Proposed System

1. Centralisation. Every piece of information a guest needs — rooms, facilities, menu, photographs, tariffs, reviews and contact details — is published in one place under the control of the business.

2. Availability. The site serves guests at any hour without consuming staff time.

3. Elimination of transcription. Because the guest enters their own name, telephone number and address into a validated field, the entire class of errors caused by transcribing speech to paper disappears.

4. Structured, durable records. Each order, payment and review becomes a row with typed columns, an automatic timestamp and a status value.

5. Two-layer validation. Client-side checks give the user instant feedback and reduce needless network traffic; server-side checks guarantee integrity even if the browser checks are bypassed.

6. Credential and payment-data protection. BCRYPT hashing makes the stored password column useless to an attacker who obtains it, and storing only the last four digits of a card removes the most sensitive field from the database entirely.

7. Persistent customer identity. Registration and session-based login establish a stable identity for each guest, which is the prerequisite for order history, saved preferences and loyalty features in later versions.

8. Systematic feedback capture. Ratings and testimonials arrive in a comparable format, which allows service quality to be tracked over time and allows the best reviews to be republished on the site.

9. Responsiveness of the interface. Asynchronous submission means the page never reloads, so the application feels quick even on a modest connection.

10. Low cost and easy maintenance. The absence of licences, frameworks and build tooling means the project can be installed, understood and modified by any developer with basic web skills, and hosted on the cheapest available shared hosting.

11. Extensibility. Because the modules are separate and the schema already contains a fully specified *bookings* table, new capabilities can be added incrementally without restructuring the existing code.

Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core processor or equivalent	Intel Core i5 / AMD Ryzen 5 or higher
Memory (RAM)	4 GB	8 GB or more
Free storage	10 GB	20 GB
Display	Standard monitor, 1366 × 768	Full HD, 1920 × 1080
Network	Local loopback access sufficient	Local network or broadband connection
Client device	Desktop or laptop computer	Desktop or laptop; tablet and mobile supported for viewing

Software Requirements

Category	Requirement
Operating system	Windows 10 or 11 (development environment); any XAMPP-compatible operating system
Web server	Apache HTTP Server, supplied by XAMPP
Server-side runtime	PHP with the MySQLi extension enabled
Database server	MySQL, supplied by XAMPP
Database administration	phpMyAdmin
Languages used	HTML, CSS, JavaScript, PHP, SQL
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox, current version
Code editor	Visual Studio Code
Package manager	None required; the project carries no dependency manifest

Functional Requirements

ID	Requirement	Input	Processing	Validation Rules
FR-01	User Registration	Full name, e-mail, phone, password, confirm password	Sanitise, validate, check for an existing e-mail, hash the password, insert into *users*	Name at least 2 characters; valid e-mail format; e-mail not already registered; phone at least 10 digits; password at least 6 characters; confirmation must match
FR-02	User Login	E-mail, password	Validate, select the user row by e-mail, verify the password against the stored hash, start a session	Valid e-mail format; password at least 6 characters; account must exist; hash must verify
FR-03	Food Ordering	Name, phone, delivery address, food item, quantity	Validate, look up the unit price on the server, multiply by quantity, insert into *orders*	Name at least 2 characters; phone at least 10 digits; address not empty; food item must exist in the server price list; quantity at least 1
FR-04	Payment Submission	Name, e-mail, address, city, ZIP code, cardholder name, card number, expiry, CVV, amount	Validate, strip non-digits from the card number, retain the last four digits only, insert into *payments* with status *success*	All fields required; valid e-mail; card number must yield at least four digits; amount greater than zero
FR-05	Review Submission	Guest name, e-mail, rating, review text	Validate, insert into *reviews*	Name at least 2 characters; valid e-mail; rating an integer between 1 and 5; review text at least 10 characters
FR-06	Website Browsing	Navigation actions	Direct page loading through hyperlinks	Not applicable
FR-07	Database Setup	MySQL credentials, local server access	Create the database if absent and create all five tables	Server must be reachable; user must hold CREATE privilege
FR-08	Session Persistence	Successful login result	Store user id, name and e-mail in *$_SESSION*	Session must be started before any output is sent

Non-Functional Requirements

Attribute	Specification
Performance	Pages are static assets with no server-side templating, so first paint is fast. Each transactional request performs a single indexed query and one insert, keeping the round trip well under one second on a local server.
Security	BCRYPT password hashing; storage limited to the last four digits of any card; server-side escaping of all input; prepared statements with bound parameters for authentication queries.
Reliability	Connection failures, invalid request methods, validation failures and duplicate registrations are all detected and answered with a structured error document rather than a fatal error.
Availability	Availability is that of the local XAMPP environment. No clustering, failover or uptime guarantee is implemented.
Usability	Short forms, explicit labels, plain-language error messages, visible confirmation of every successful action, and a consistent navigation bar on every page.
Maintainability	One PHP file per responsibility, a single shared connection file, and consistent naming across the front end, the back end and the schema.
Scalability	Adequate for a single-property workload. Connection pooling, caching, indexing strategy and asynchronous job handling are not implemented.
Compatibility	Verified on current versions of Chrome, Edge and Firefox. Only standard browser APIs are used.
Responsiveness	Flexible grids and CSS media rules adapt the layout to desktop, tablet and mobile widths.

3.2 Development Strategy Model

The project was developed using an **iterative model with waterfall-style phases**. The overall sequence of phases — requirement analysis, design, implementation, testing and deployment — follows the classical waterfall order, because the requirements of a hotel website were stable and well understood from the beginning and did not need to be discovered through experimentation. Within that sequence, however, each module was carried through the full cycle on its own before the next module began, which gives the process its iterative character.

1. **Phase 1 — Requirement Analysis.** The existing manual workflow was studied and the deficiencies listed in Section 2.2 were identified. From these, the functional requirements FR-01 to FR-08 and the non-functional attributes of Section 3.1 were derived.
2. **Phase 2 — Planning.** The page inventory was agreed, the navigation structure was drawn, and the modules were ordered by dependency. Because ordering and payment depend on a working database connection, and payment depends on ordering, the sequence was fixed as: static pages, database schema, connection layer, registration, login, ordering, payment, reviews.
3. **Phase 3 — Design.** The visual identity was settled first: a gold accent on a deep navy base, a card-based grid for every collection of items, and a full-width hero banner on each major page. In parallel the database was designed, producing the five tables and the column definitions documented in Section 4.7.
4. **Phase 4 — Development.** Implementation began with the static presentation pages, which allowed the layout and styling to stabilise before any dynamic behaviour was added. The

shared connection file was written next, followed by the setup scripts that create the schema.

5. **Phase 5 — Testing.** Each module was tested manually as soon as it was completed, using both valid and invalid data, and the resulting database state was verified through phpMyAdmin. When all modules were in place a second round of testing exercised the complete guest journey from browsing through registration, login, ordering, payment and review.
6. **Phase 6 — Deployment.** Deployment consists of copying the project directory into the XAMPP web root, starting Apache and MySQL, running the setup script once to create the schema, and opening the site in a browser. No build step, compilation or dependency installation is required.
7. **Phase 7 — Maintenance.** Maintenance is performed by editing source files directly. Content changes such as a new room or a revised tariff are made in the HTML; pricing changes are made in the server-side price list in *order.php*; schema changes are applied through phpMyAdmin or by extending the setup scripts.

Justification of the model. A pure waterfall model was unsuitable because it would have deferred all testing to the end, by which time an error in the connection layer would have contaminated every module built on top of it. A fully agile model was equally unsuitable, since the project had a single stakeholder, a fixed deliverable and a fixed academic deadline, and therefore no need for sprint ceremonies or continuous re-prioritisation.

4- System Design

4.1 Data Flow Diagram

The data-flow diagram describes how information moves between the external entities, the processes of the system and the stored data collections. The context diagram presents the system as a single process, and the level-1 diagram decomposes that process into the five transactional operations that the system actually performs.

Context Level Diagram (Level 0)

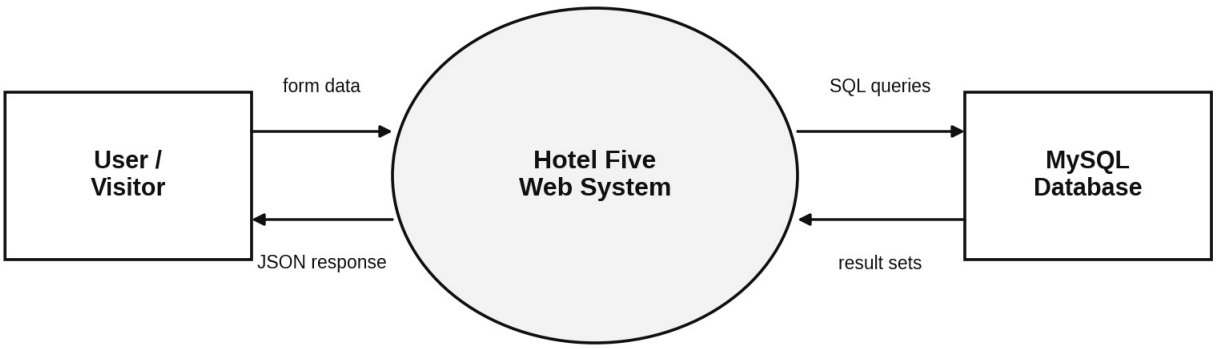


Fig. 4.1.1 Context Level (Level 0) Data Flow Diagram

The context diagram treats the whole website as one process. The user sends form data (registration, login, order, payment and review details) into the system and receives a JSON response. Internally the system exchanges SQL queries and result sets with the MySQL database *food_website*.

Level 1 Data Flow Diagram

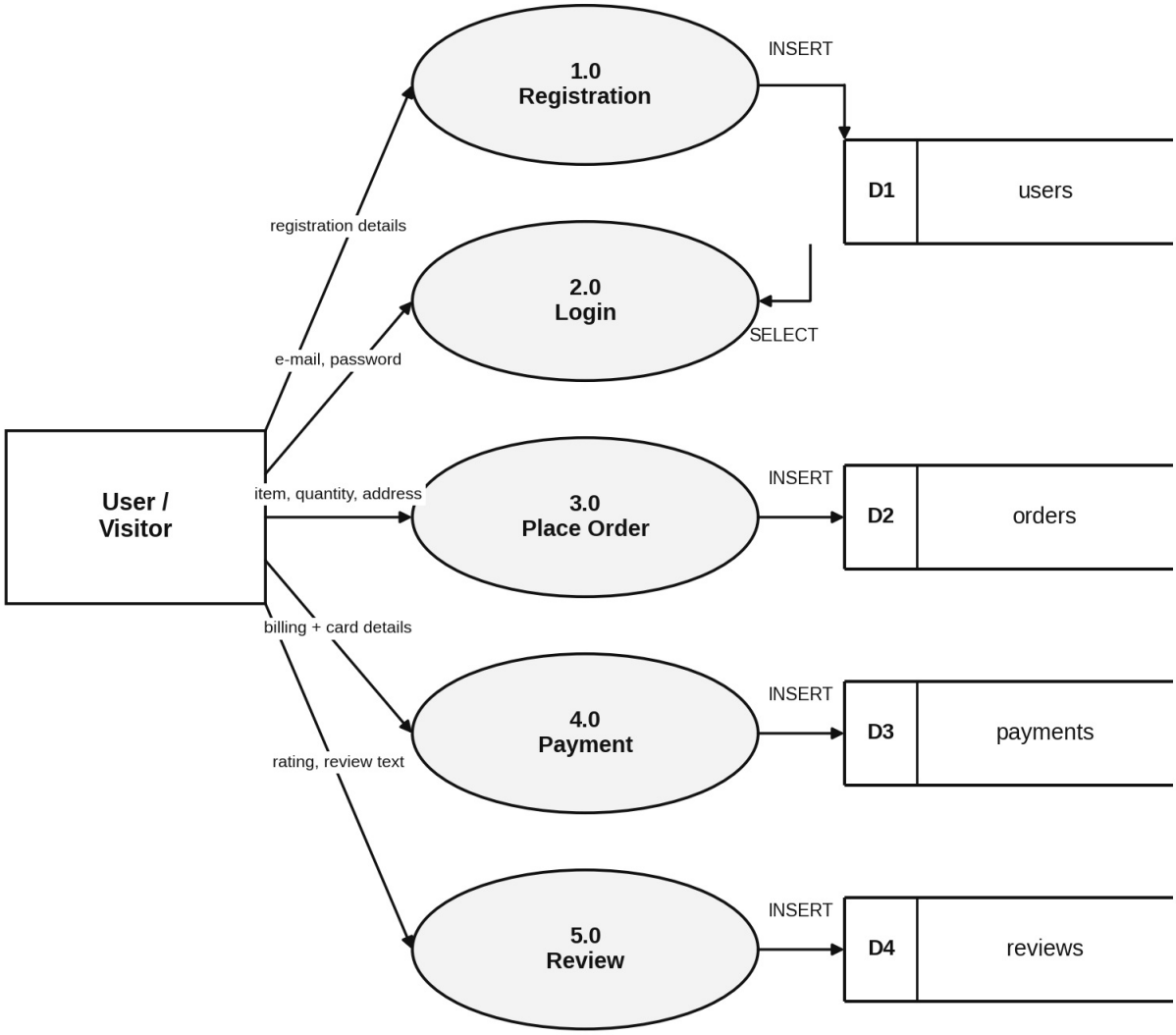
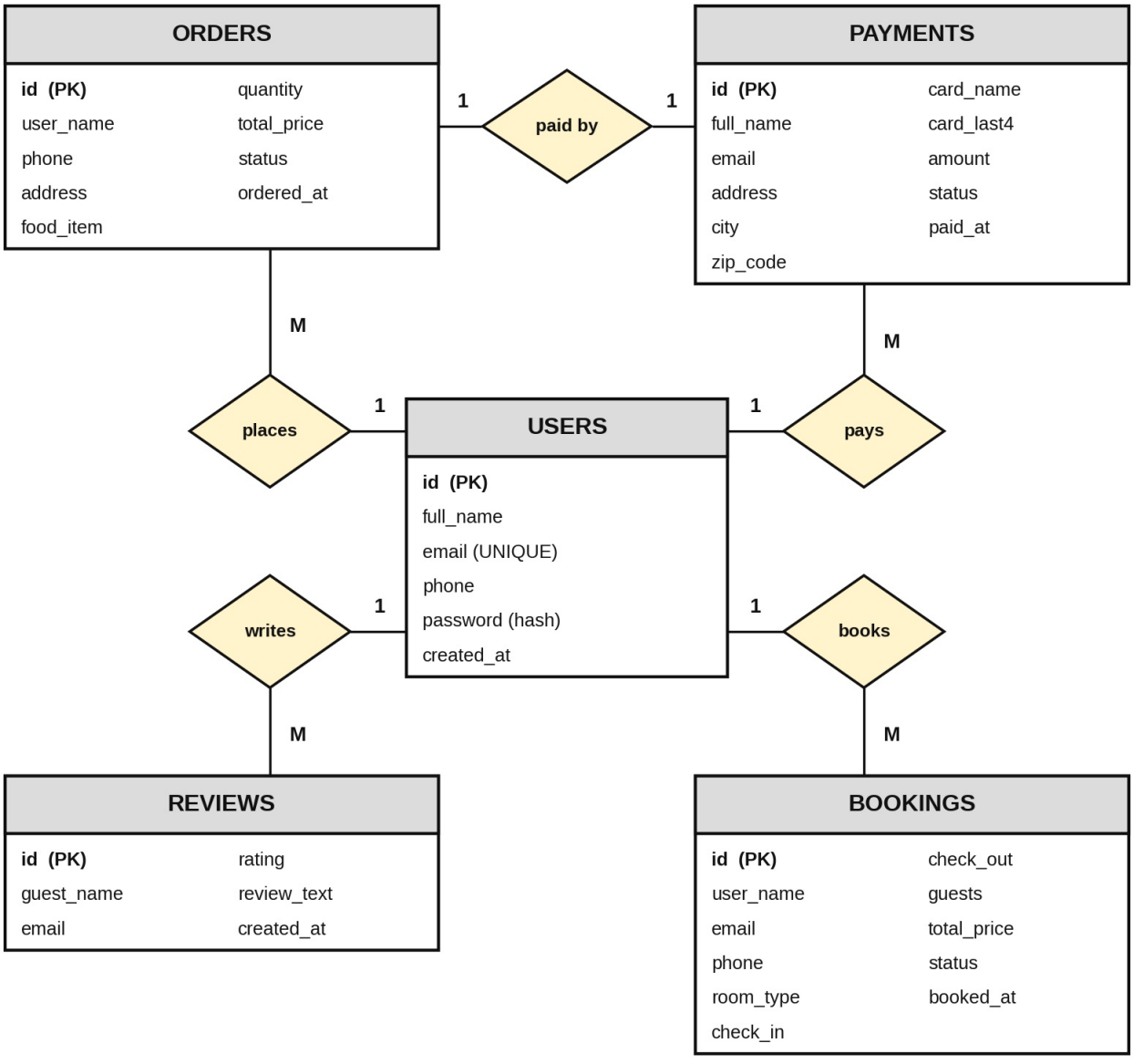


Fig. 4.1.2 Level 1 Data Flow Diagram (every process also returns a JSON status to the user)

The level-1 diagram shows that each of the five transactional processes receives its input directly from the user and writes to exactly one data store. Only the login process reads from a store rather than writing to it, since it must retrieve the stored password hash in order to verify the supplied credentials. Every process also returns a JSON status document to the user; these return flows are omitted from the diagram for clarity, as they are identical in every case.

4.2 E-R Diagram

The entity-relationship diagram presents the logical structure of the database. Five entities are defined. The *users* entity is conceptually the parent of the other four, since orders, payments, reviews and bookings are all created by a person. It must be recorded honestly that these relationships are conceptual: the implementation does not declare foreign-key constraints, because the order, payment and review forms are open to guests who have not logged in, and a mandatory foreign key would have made anonymous submission impossible.



PK = primary key. Relationships are logical (no foreign-key constraints are declared), because orders, payments and reviews can be submitted by guests without an account. The bookings table is prepared for the room-booking workflow of a future version.

Fig. 4.2 Entity Relationship Diagram of database food_website

4.3 Class Diagram

The project is written in procedural PHP rather than with object-oriented classes. The class diagram below is therefore a logical model: each back-end script is represented as the class it would become if the code were refactored into objects, and the operations listed are the actual functional steps performed inside that script. This representation is useful because it makes the responsibilities of each module explicit and shows clearly that every controller depends on the single shared connection component.

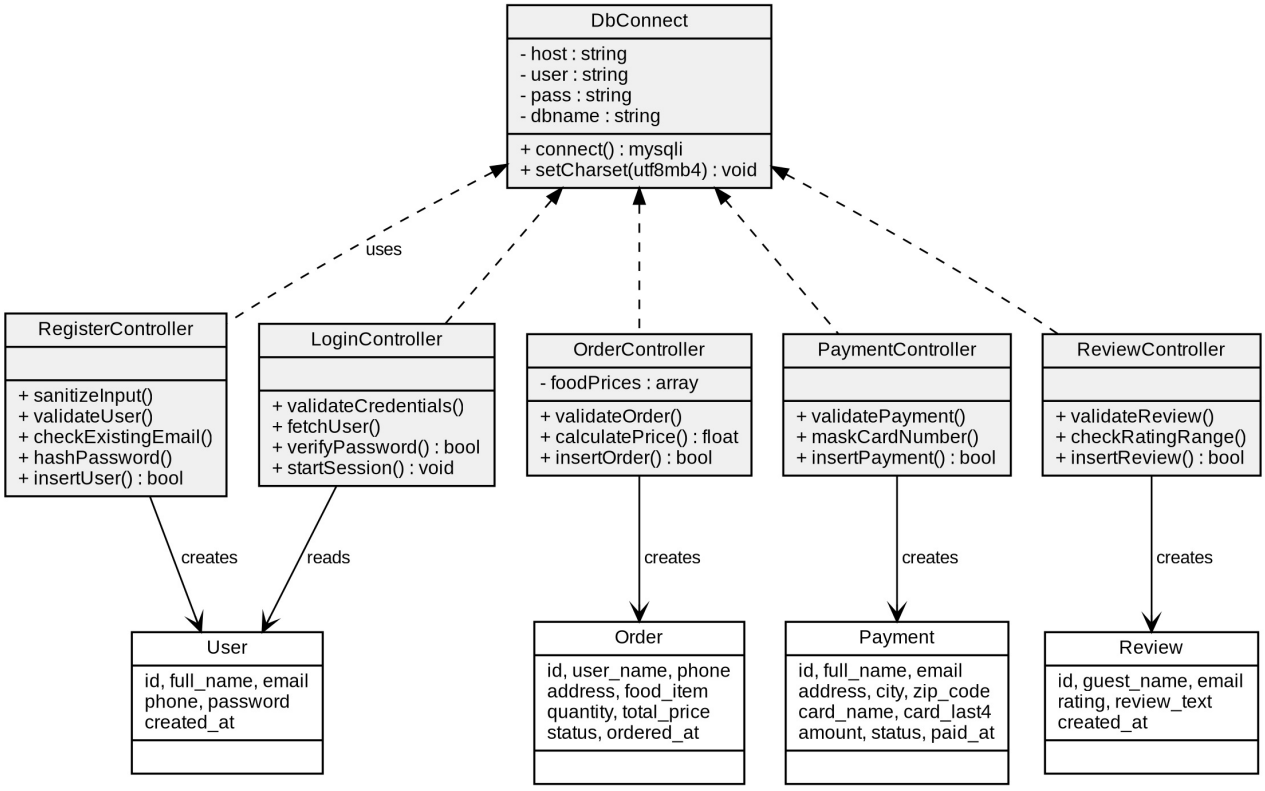


Fig. 4.3 Class Diagram (logical model of the PHP back end)

In the diagram, a dashed arrow towards *DbConnect* means that the controller depends on the shared connection file *db_connect.php*. A solid arrow from a controller to an entity means that the controller creates (or, for login, reads) records of that entity. The *bookings* entity has no controller yet because the room-booking workflow is a future enhancement.

4.4 Use Case Diagram

The use-case diagram identifies the actors who interact with the system and the services each of them may invoke. Three human actors are shown. The **Visitor** is an unauthenticated guest who can browse every public page and register an account. The **Registered User** is a specialisation of the visitor (shown with a generalisation arrow), so a registered user can do everything a visitor can, and additionally logs in, places food orders, pays and submits reviews. Placing an order *includes* the payment step, because a successful order hands the user over to the checkout page. The **Administrator** is drawn with dashed lines to show that the role is designed but not implemented in this release.

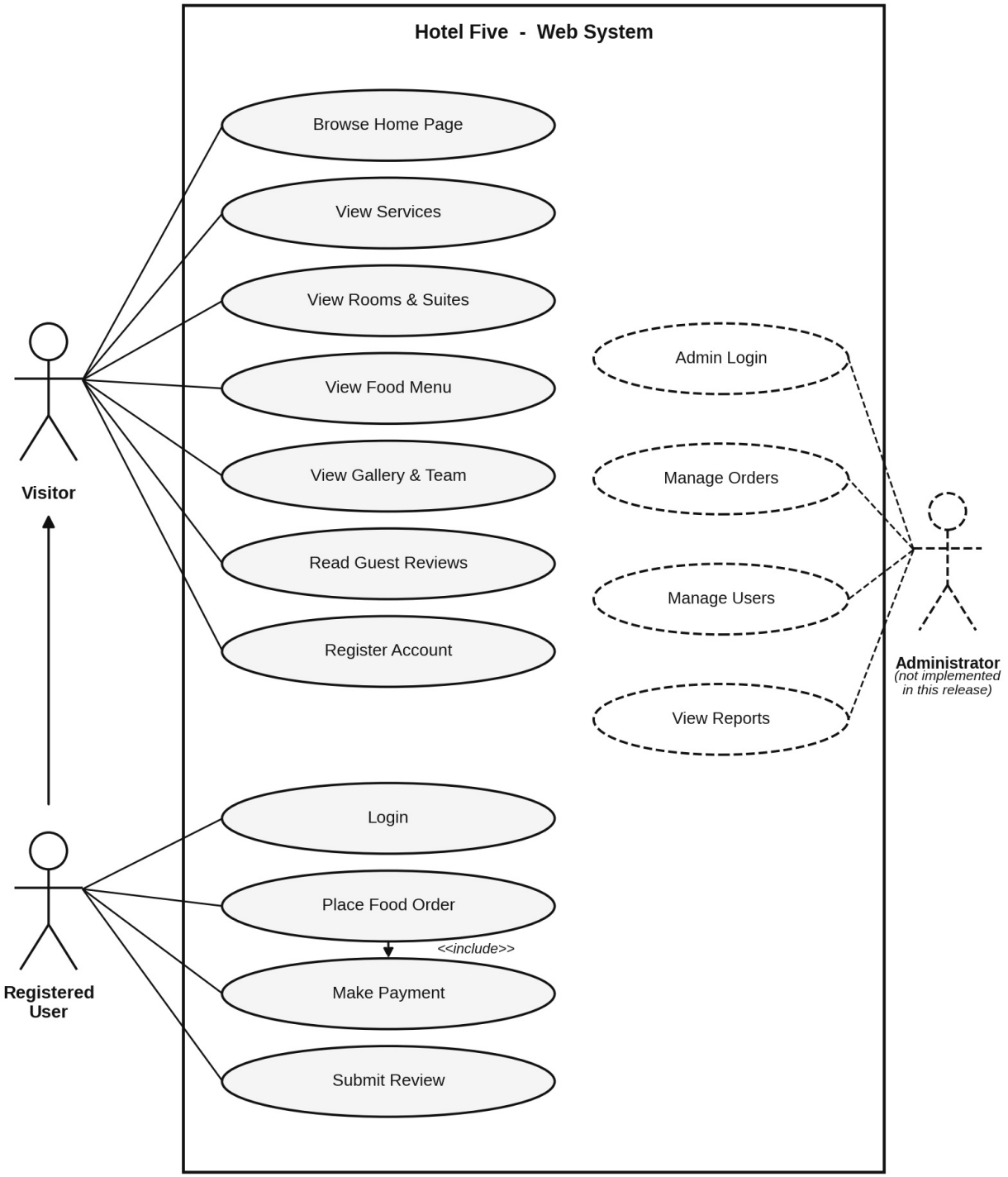


Fig. 4.4 Use Case Diagram

4.5 Sequence Diagram

Sequence diagrams show the order in which messages pass between the participants of a workflow. Four workflows are shown: registration, login, placing an order and payment. In every one of them the browser validates the form first, sends the data asynchronously with *fetch()*, and the PHP script validates it again, talks to MySQL and answers with a JSON document.

Registration Sequence

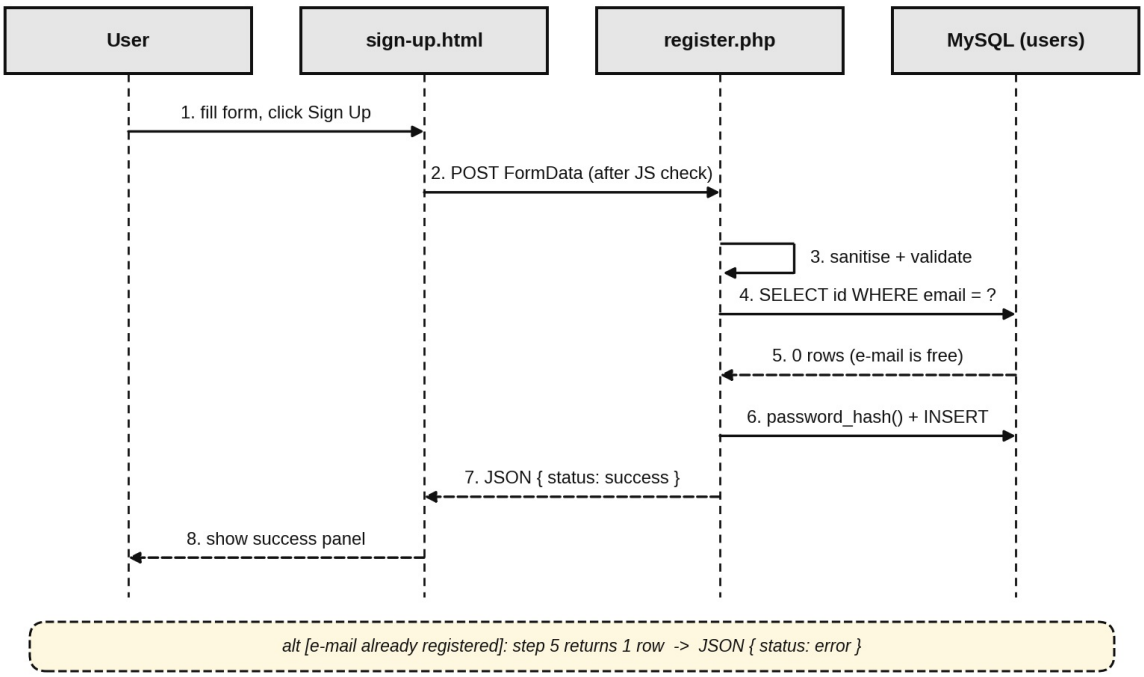


Fig. 4.5.1 Sequence diagram - User Registration

Login Sequence

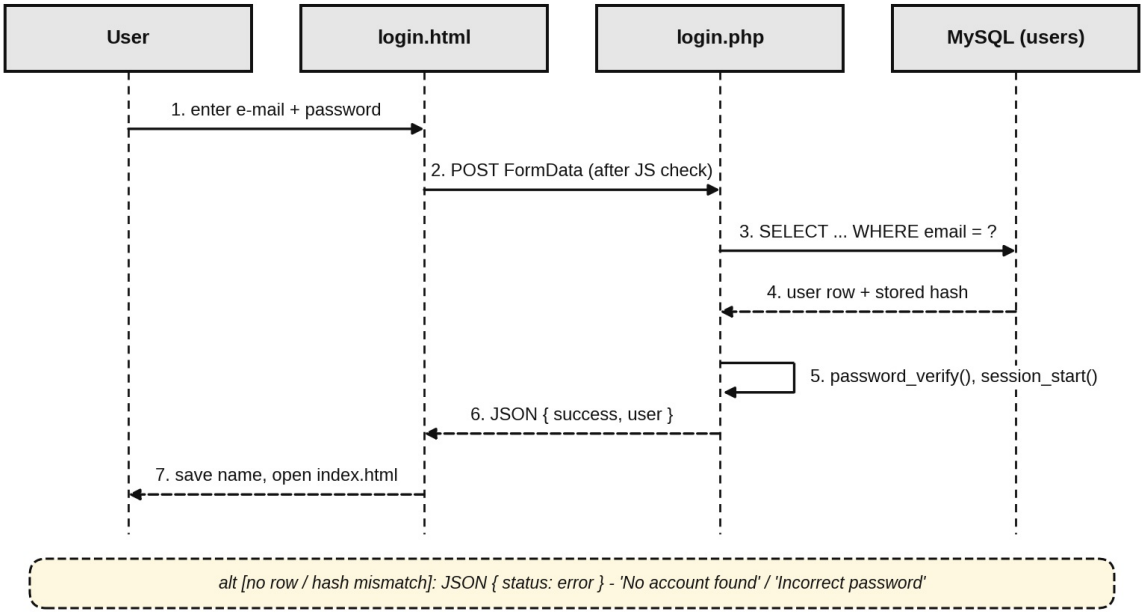


Fig. 4.5.2 Sequence diagram - User Login

Order Sequence

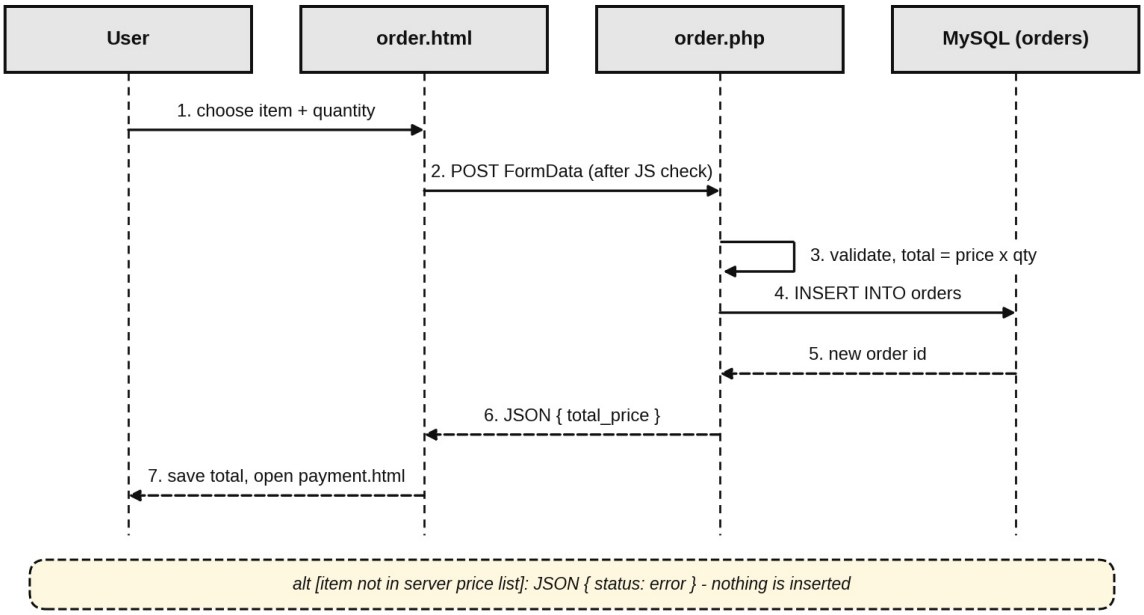


Fig. 4.5.3 Sequence diagram - Placing a Food Order

Payment Sequence

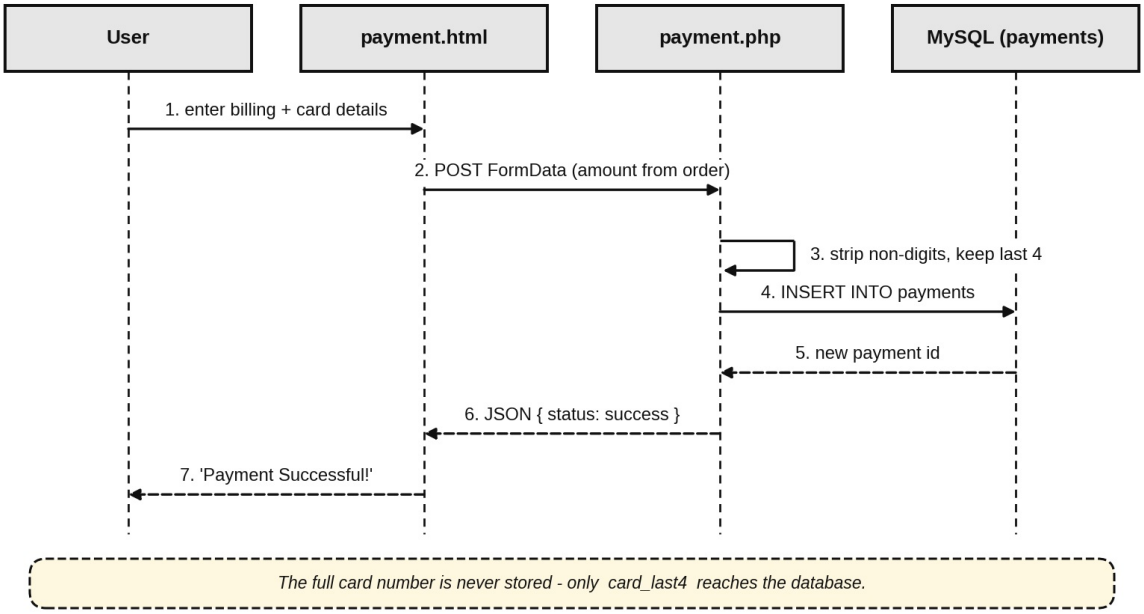


Fig. 4.5.4 Sequence diagram - Payment

4.6 Activity Diagram

The activity diagram traces the complete guest journey through the system, from arriving at the home page to receiving confirmation of a payment, including the decision points at which the flow may branch to an error state.

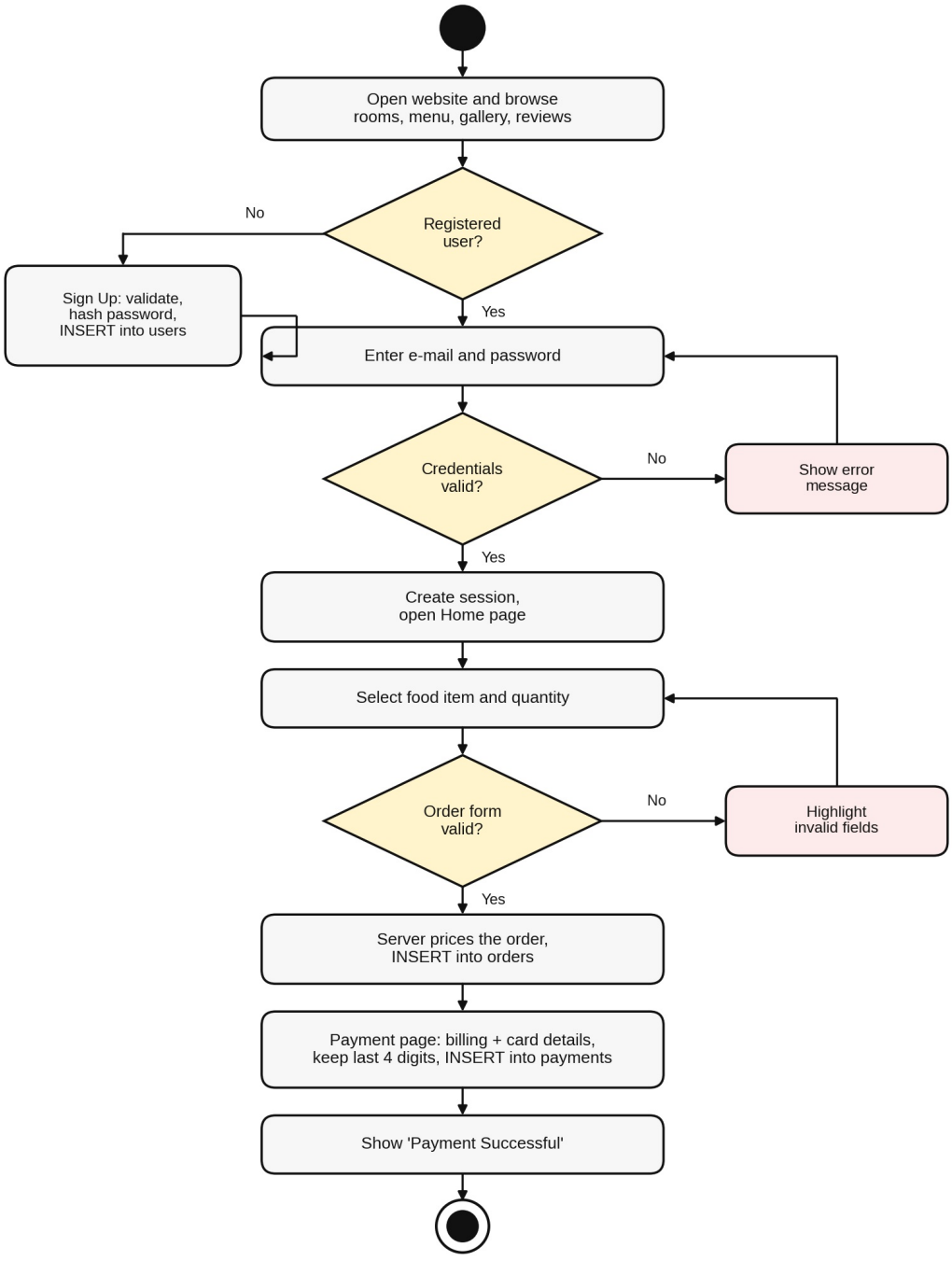


Fig. 4.6 Activity Diagram of the complete guest journey

4.7 Data Dictionary (Database)

The database is named **food_website** and contains five tables. Each table is described below with its complete column definition, data type, size, constraints and purpose. The character set used throughout is utf8mb4, which is applied on the connection in *db_connect.php*.

Table 1 — users

Purpose: stores registered account information created through the sign-up form and read during authentication.

#	Field Name	Data Type	Size	Constraints	Description
1	id	INT	—	PRIMARY KEY, AUTO_INCREMENT	Unique identifier of the user
2	full_name	VARCHAR	100	NOT NULL	Full name of the account holder
3	email	VARCHAR	150	NOT NULL, UNIQUE	E-mail address, used as the login identifier
4	phone	VARCHAR	20	NOT NULL	Contact number, minimum ten digits
5	password	VARCHAR	255	NOT NULL	BCRYPT hash of the password; never stored in plain text
6	created_at	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Date and time of registration

Table 2 — orders

Purpose: stores every food order placed through the order page, with the total priced on the server.

#	Field Name	Data Type	Size	Constraints	Description
1	id	INT	—	PRIMARY KEY, AUTO_INCREMENT	Unique identifier of the order
2	user_name	VARCHAR	100	NOT NULL	Name of the customer placing the order
3	phone	VARCHAR	20	NOT NULL	Contact number for delivery
4	address	VARCHAR	255	NOT NULL	Delivery address
5	food_item	VARCHAR	100	NOT NULL	Selected menu item
6	quantity	INT	—	NOT NULL, DEFAULT 1	Number of units ordered, minimum 1
7	total_price	DECIMAL	10,2	NOT NULL, DEFAULT 0.00	Unit price multiplied by quantity, computed on the server
8	status	ENUM	—	DEFAULT 'pending'	One of pending, confirmed, delivered, cancelled
9	ordered_at	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Date and time the order was placed

Table 3 — payments

Purpose: stores billing information and the masked card reference for each checkout.

#	Field Name	Data Type	Size	Constraints	Description
1	id	INT	—	PRIMARY KEY, AUTO_INCREMENT	Unique identifier of the payment
2	full_name	VARCHAR	100	NOT NULL	Name on the billing record
3	email	VARCHAR	150	NOT NULL	E-mail address for the receipt
4	address	VARCHAR	255	NOT NULL	Billing street address
5	city	VARCHAR	100	NOT NULL	Billing city
6	zip_code	VARCHAR	20	NOT NULL	Postal code
7	card_name	VARCHAR	100	NOT NULL	Name printed on the card
8	card_last4	CHAR	4	NOT NULL	Final four digits only; the full number is never stored
9	amount	DECIMAL	10,2	NOT NULL, DEFAULT 2290.00	Amount payable, carried from the order total
10	status	ENUM	—	DEFAULT 'pending'	One of pending, success, failed
11	paid_at	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Date and time of the transaction

Table 4 — reviews

Purpose: stores guest feedback submitted through the review form.

#	Field Name	Data Type	Size	Constraints	Description
1	id	INT	—	PRIMARY KEY, AUTO_INCREMENT	Unique identifier of the review
2	guest_name	VARCHAR	100	NOT NULL	Name of the reviewer, minimum two characters
3	email	VARCHAR	150	NOT NULL	E-mail address of the reviewer
4	rating	TINYINT	—	NOT NULL, DEFAULT 5	Star rating, permitted range 1 to 5
5	review_text	TEXT	—	NOT NULL	Review body, minimum ten characters
6	created_at	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Date and time the review was submitted

Table 5 — bookings

Purpose: defines the structure required for room reservations. The schema is complete, but the

corresponding front-end workflow has not been connected in this release, so the table exists as prepared groundwork for the next iteration.

#	Field Name	Data Type	Size	Constraints	Description
1	id	INT	—	PRIMARY KEY, AUTO_INCREMENT	Unique identifier of the booking
2	user_name	VARCHAR	100	NOT NULL	Name of the guest
3	email	VARCHAR	150	NOT NULL	Contact e-mail address
4	phone	VARCHAR	20	NOT NULL	Contact number
5	room_type	VARCHAR	100	NOT NULL	Selected room or suite category
6	check_in	DATE	—	NOT NULL	Arrival date
7	check_out	DATE	—	NOT NULL	Departure date
8	guests	INT	—	NOT NULL, DEFAULT 1	Number of guests
9	total_price	DECIMAL	10,2	NOT NULL, DEFAULT 0.00	Computed tariff for the stay
10	status	ENUM	—	DEFAULT 'pending'	One of pending, confirmed, cancelled
11	booked_at	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Date and time the booking was created

Note on Referential Integrity

No foreign-key constraint is declared between *users* and the transactional tables. This is a deliberate design decision rather than an omission: the order, payment and review forms are open to guests who have not created an account, and a mandatory foreign key would have made anonymous submission impossible. The relationship is therefore maintained logically through the name and e-mail columns. If a future version restricts these actions to authenticated users, a *user_id* column with a proper foreign-key constraint should be added to each of the four child tables.

5- Screen Design

This chapter documents every screen of the “Hotel Five” application. For each screen the actual interface is shown, followed by an explanation of its purpose, its layout, the elements it contains, the input it accepts and the role it plays within the overall system. Section 5.1 covers the input design, that is, the screens through which the user supplies information or navigates. Section 5.2 covers the output design, that is, the messages, confirmations and stored records the system produces in response.

Screen Inventory

Screen	File	Primary Purpose
Home	index.html	Brand presentation, hero banner, quick-search bar and entry point to every module
Services	index.html (section)	Overview of the six facilities offered by the property
About Us	about.html	Company story, differentiators, achievement statistics and photo strip
Our Rooms	room.html, room-1 to room-8.html	Room catalogue with tariffs, capacity, amenities and booking action
Our Menu	our_food.html, burger.html	Restaurant menu with descriptions, ratings, prices and order action
Gallery	galary.html	Photographic record of the property
Popular Hotels	popular.html	Highlighted group properties
Team	team.html	Introduction to the staff who serve the guest
Reviews	review.html	Display of guest testimonials and the review submission form
Order Food	order.html	Collection of order and delivery details
Payment	payment.html	Collection of billing identity and card details
Login	login.html	Authentication of returning users
Sign Up	sign-up.html	Creation of a new user account

*Note: the project name is **Hotel Five** (see the logo in the navigation bar of every screen). A few headings and sub-lines on the pages, such as “A Glimpse of Luxe” and “Sign in to your Luxe account”, still carry the earlier working title and are quoted below exactly as they appear on screen.*

5.1 Input Design

5.1.1 Home Page :

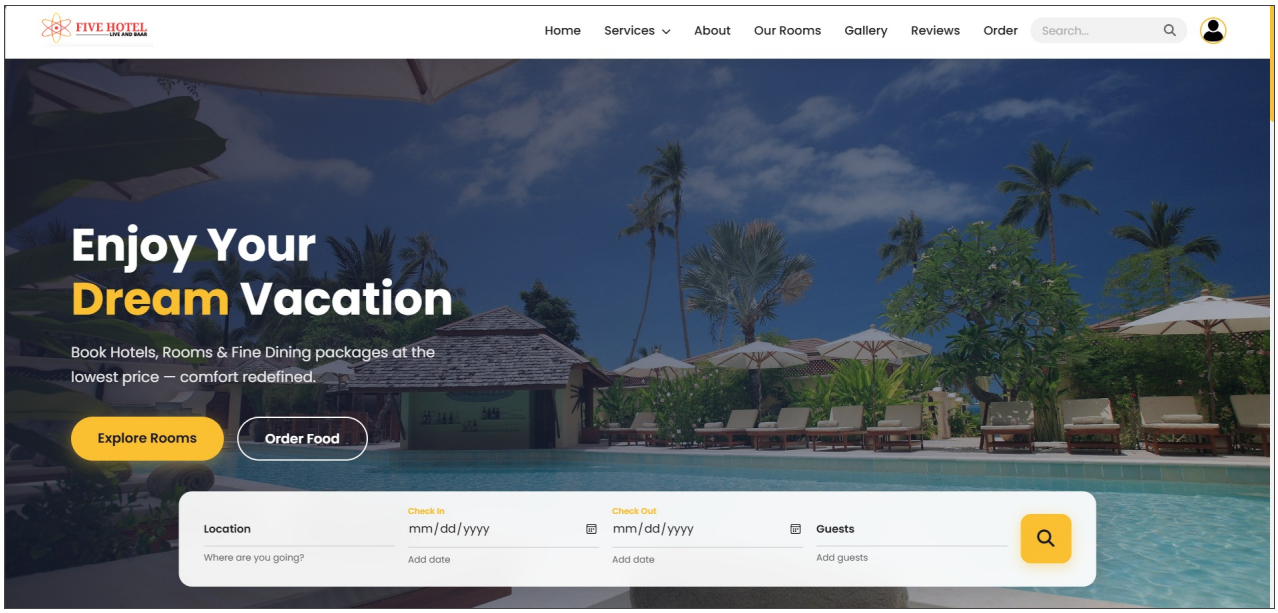


Fig. 5.1.1 — Home Page (index.html)

Description :

The home page is the first screen a visitor encounters and therefore carries the largest share of the responsibility for forming an impression of the property. It opens with a fixed navigation bar that stays visible as the page scrolls. The bar holds the brand logo on the left and the primary links on the right: Home, Services with a dropdown, About, Our Rooms, Gallery, Reviews and Order, followed by a search field and an account icon.

Below the bar is a full-width hero banner showing a resort pool surrounded by palms. Over this image sits the headline “Enjoy Your Dream Vacation”, with the word *Dream* rendered in the gold accent colour that identifies the brand throughout the site. A supporting line beneath it states the proposition in a single sentence: hotels, rooms and fine-dining packages at the lowest price.

Resting on the lower edge of the hero is the quick-search panel, a white card containing four inputs — Location, Check In, Check Out and Guests — with a gold search button. Placing this panel so that it overlaps the banner is a deliberate layout decision: it draws the eye downward and signals that the site is transactional rather than merely informational. Each field carries a helper line such as “Where are you going?” or “Add date” so that its meaning is unambiguous. It must be stated for accuracy that in the present release this panel is a presentational element; the search back end is listed in Chapter 8 as a planned enhancement.

5.1.2 Services Section :

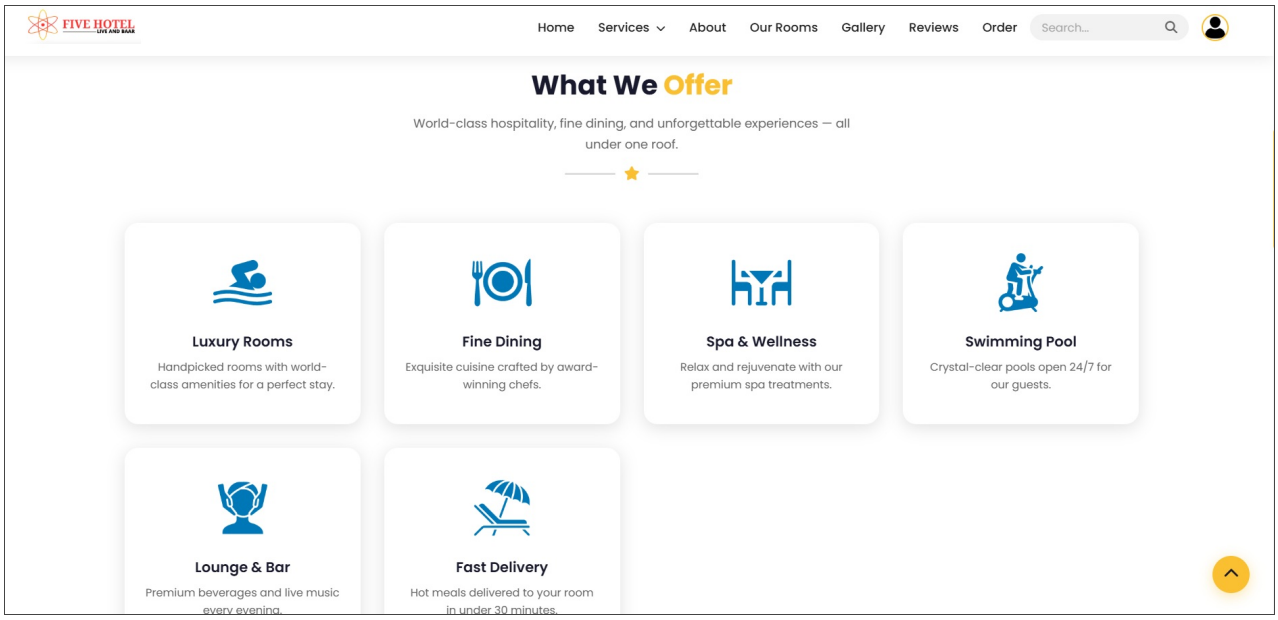


Fig. 5.1.2 — What We Offer (Services)

Description :

The services section answers the question a guest asks immediately after deciding that the property looks attractive: what exactly do I get here? It is introduced by the heading “What We Offer” with the accent word in gold, a one-line summary describing world-class hospitality, fine dining and memorable experiences under one roof, and a small star divider that separates the heading block from the content below.

The facilities are presented as six cards laid out in a responsive grid, four to a row on a desktop display and fewer as the viewport narrows. Each card contains a Font Awesome icon rendered in blue, a bold title and a one-sentence description. The six facilities are **Luxury Rooms**, handpicked rooms with world-class amenities; **Fine Dining**, cuisine prepared by award-winning chefs; **Spa & Wellness**, premium treatments for relaxation; **Swimming Pool**, pools open around the clock; **Lounge & Bar**, premium beverages with live music each evening; and **Fast Delivery**, hot meals delivered to the room within thirty minutes.

5.1.3 About Us Page :

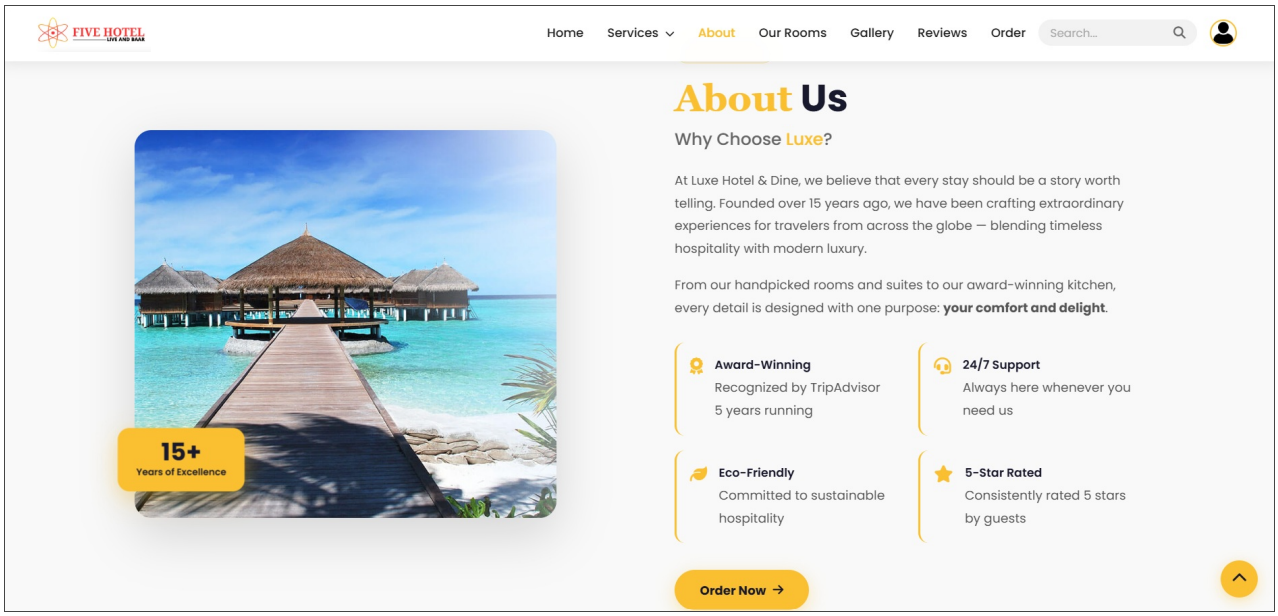


Fig. 5.1.3 — About Us (about.html)

Description :

The About page exists to convert interest into trust. Where the home page shows what the property looks like and the services section states what it provides, this page explains who is behind it and why a guest should believe the claims made elsewhere on the site.

The layout is a two-column arrangement. On the left is a photograph of over-water villas against turquoise water, with a gold badge overlapping its lower-left corner reading **15+ Years of Excellence**. Overlapping the image with the badge rather than placing it alongside makes the claim read as part of the picture rather than as a separate assertion, which is a common and effective device in hospitality marketing.

5.1.4 Statistics and Photo Strip :

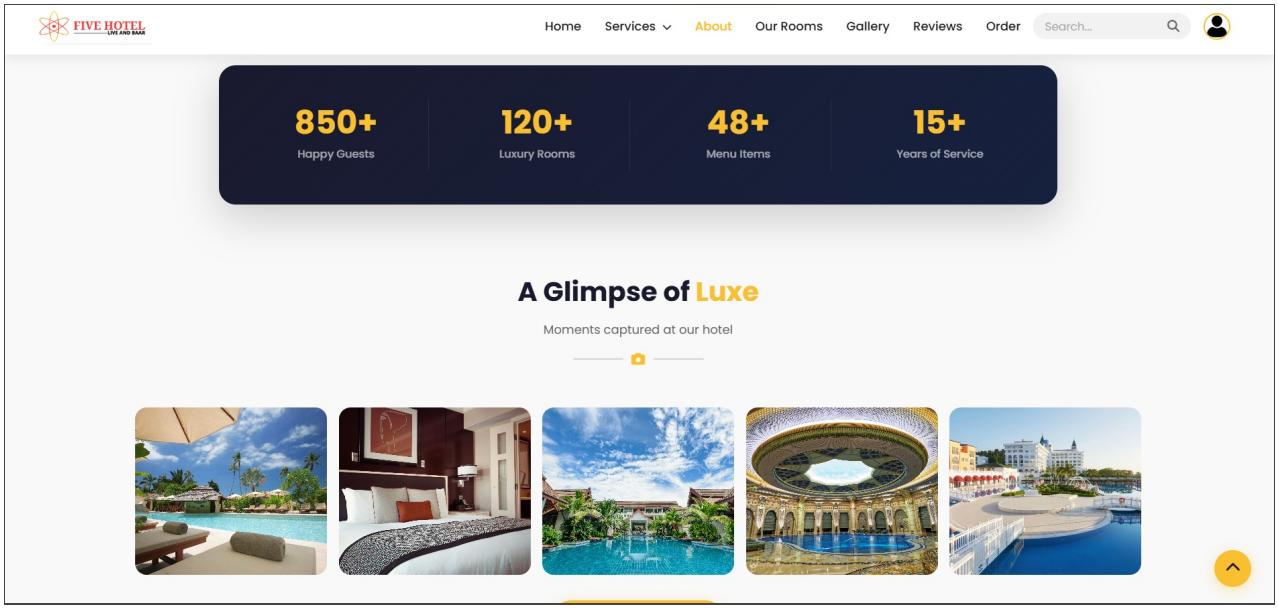


Fig. 5.1.4 — Achievement Statistics and "A Glimpse of Luxe"

Description :

This screen contains two distinct components that together close the About page. The first is the

statistics band, a deep navy panel with rounded corners carrying four figures in large gold type: **850+** happy guests, **120+** luxury rooms, **48+** menu items and **15+** years of service. Thin vertical rules separate the four so that each reads as an independent fact.

Below the band is the photo strip headed “A Glimpse of Luxe” with the sub-line “Moments captured at our hotel” and a camera icon divider. Five photographs run across the full width of the content area at equal height: a poolside lounging area, a furnished bedroom, an open-air pool under a blue sky, an ornate domed indoor pool and a waterside resort facade. The strip is deliberately varied in subject so that a visitor can see accommodation, leisure and architecture in a single glance, and it acts as a bridge to the full Gallery page for anyone who wants to look further.

5.1.5 Our Rooms Page :

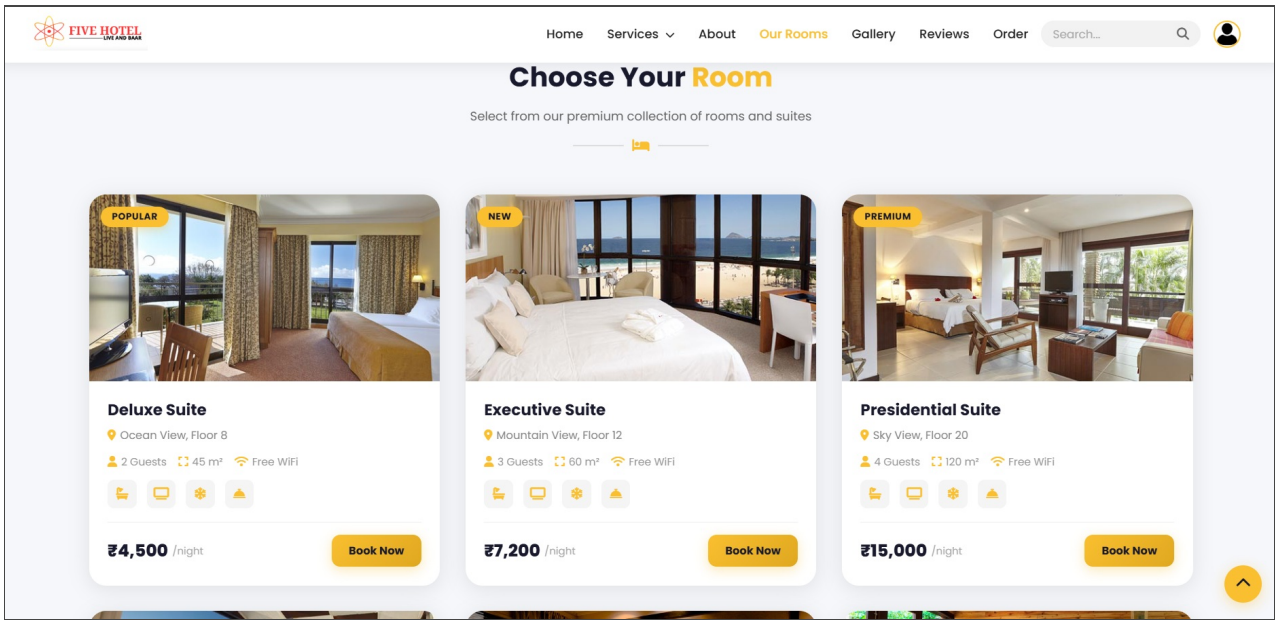


Fig. 5.1.5 — Choose Your Room (room.html)

Description :

The rooms page is the principal catalogue screen of the accommodation side of the business. It is headed “Choose Your Room” with the instruction to select from the premium collection of rooms and suites, followed by a bed-icon divider.

Rooms are presented as cards in a three-column grid. Each card is a complete, self-sufficient summary of one room, and every card carries exactly the same information in exactly the same order so that a guest can compare options without re-reading. Reading from the top, a card contains a photograph of the room; a category badge in the top-left corner (**POPULAR**, **NEW** or **PREMIUM**); the room name; a location line with a pin icon giving the view and the floor; a specification line with three items, guest capacity, floor area in square metres and connectivity; a row of four amenity icons; and finally a footer containing the nightly tariff in rupees and a gold **Book Now** button.

5.1.6 Our Menu Page :

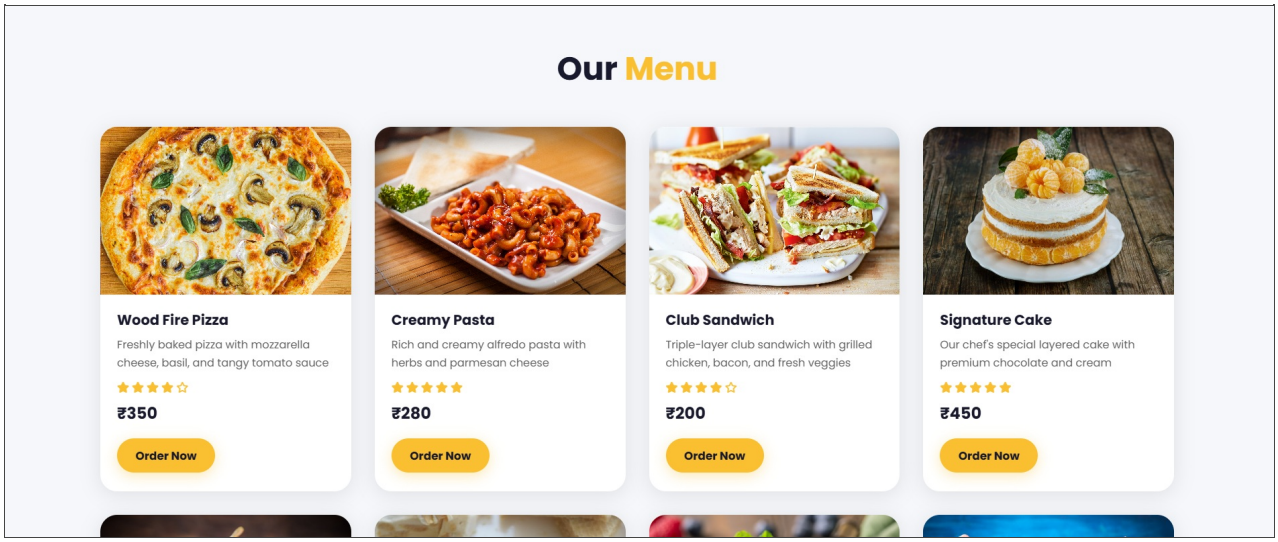


Fig. 5.1.6 — Our Menu (our_food.html)

Description :

The menu screen is the catalogue for the dining side of the business and is structured in deliberate parallel with the rooms page, so that a guest who has understood one immediately understands the other. It is headed simply “Our Menu” with the accent word in gold.

Each dish occupies a card containing a large photograph, the dish name, a one-line description, a star rating and the price in rupees, closing with a gold **Order Now** button. The four items visible are **Wood Fire Pizza** at ₹350, freshly baked with mozzarella, basil and tomato sauce, rated four stars; **Creamy Pasta** at ₹280, alfredo with herbs and parmesan, rated five stars; **Club Sandwich** at ₹200, a triple-layer sandwich with grilled chicken, bacon and vegetables, rated four stars; and **Signature Cake** at ₹450, the chef’s layered cake with premium chocolate and cream, rated five stars. Further rows continue below with the remaining items of the menu.

5.1.7 Gallery Page :

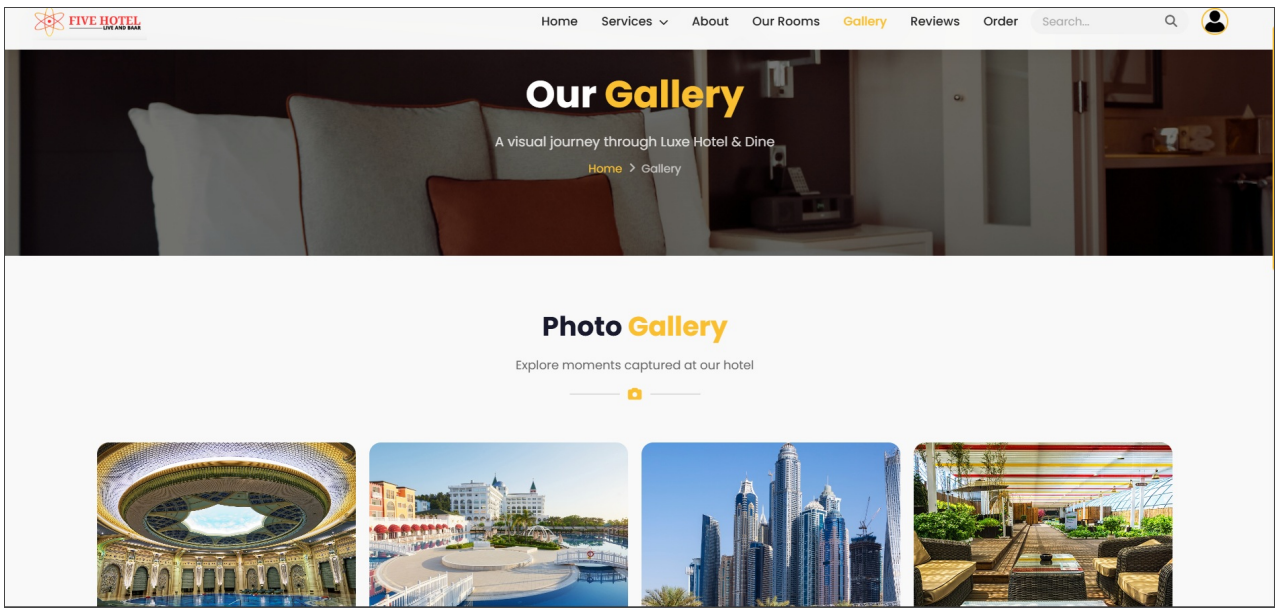


Fig. 5.1.7 — Our Gallery (galary.html)

Description :

The gallery page satisfies the guest who has read the descriptions and now wants visual evidence.

It opens with a full-width banner showing a made-up bed in warm light, overlaid with the heading “Our Gallery”, the sub-line “A visual journey through Luxe Hotel & Dine” and a breadcrumb trail reading Home › Gallery. The breadcrumb appears on every secondary page of the site and tells the visitor at a glance where they are within the structure.

Below the banner the content area is headed “Photo Gallery” with the sub-line “Explore moments captured at our hotel” and a camera divider. Photographs are arranged in a uniform grid with rounded corners and consistent spacing. The visible row shows an ornate domed indoor pool, a waterside resort facade with a circular pool, a city skyline of high-rise towers and a covered garden terrace with wicker seating.

5.1.8 Popular Hotels Page :

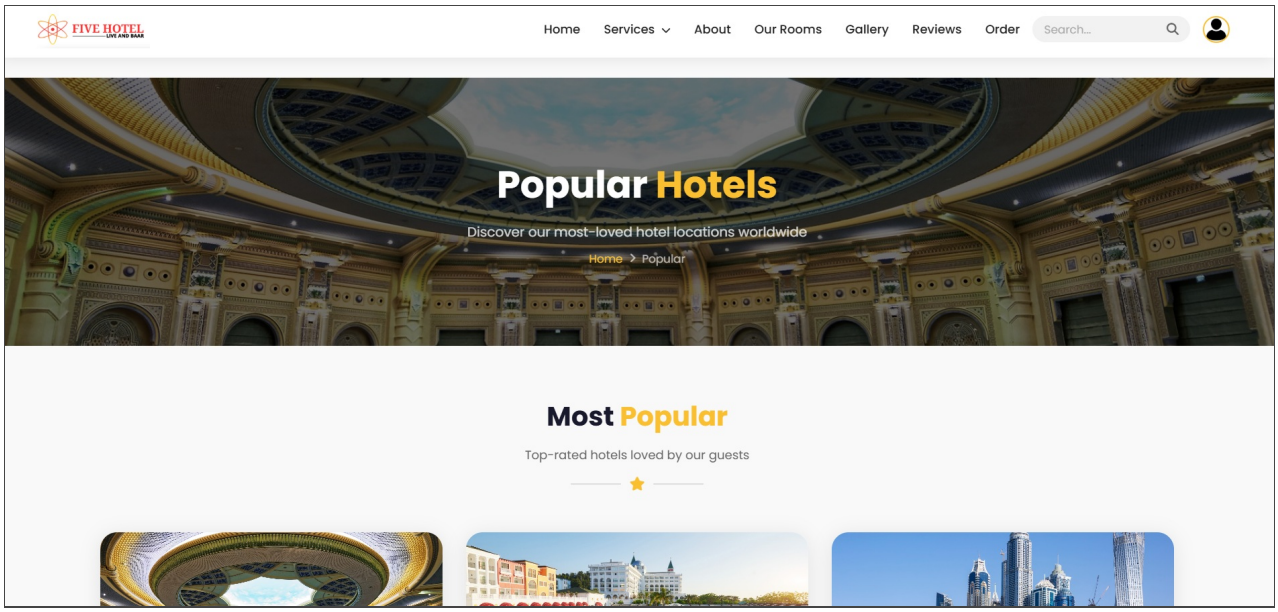


Fig. 5.1.8 — Popular Hotels (popular.html)

Description :

The Popular Hotels page presents the group’s most-loved properties and positions the brand as a chain rather than a single establishment. Its banner shows an ornate domed ceiling and carries the heading “Popular Hotels”, the sub-line describing the most-loved locations worldwide, and the breadcrumb Home › Popular.

The content area is headed “Most Popular” with the sub-line “Top-rated hotels loved by our guests” and a star divider, followed by a grid of property cards each carrying a photograph of the location. The page serves two commercial purposes. First, it broadens the choice available to a guest whose plans are not fixed to a single city.

5.1.9 Team Page :

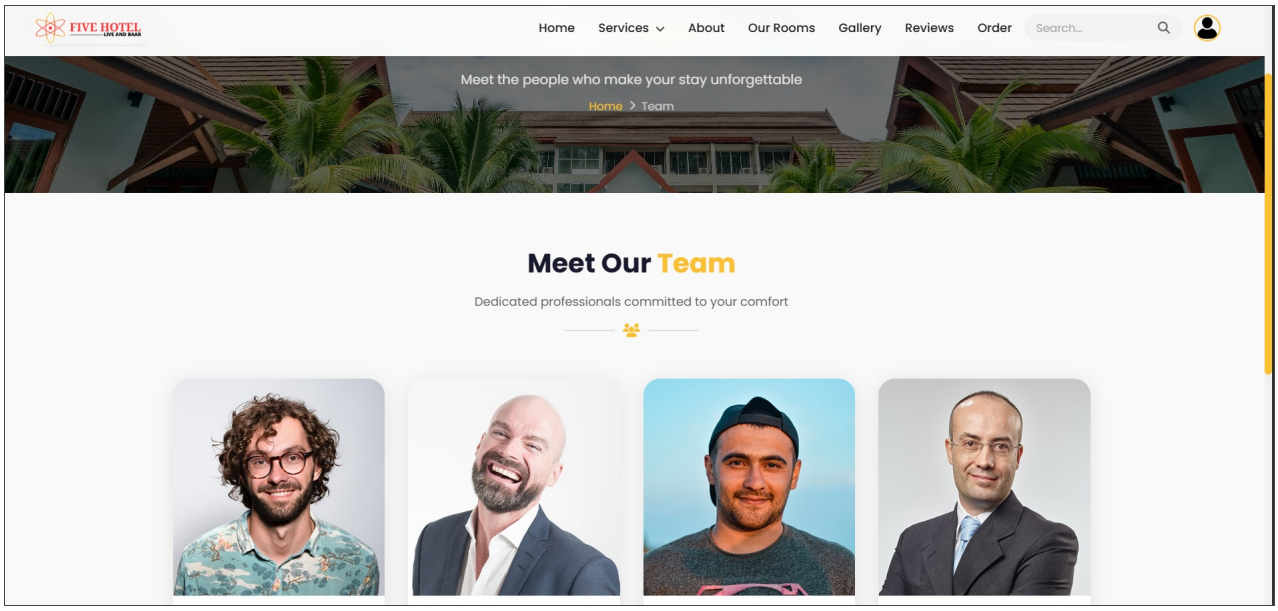


Fig. 5.1.9 — Meet Our Team (team.html)

Description :

The team page introduces the people who deliver the service. Its banner shows the property behind palm trees and carries the line “Meet the people who make your stay unforgettable” together with the breadcrumb Home › Team. The content area is headed “Meet Our Team” with the sub-line “Dedicated professionals committed to your comfort” and a people-icon divider.

Staff members are presented as cards in a four-column grid, each containing a portrait photograph with the staff member’s details shown beneath the photograph. The page exists because hospitality is ultimately a service delivered by people, and a guest who has seen the faces of the staff arrives with a different expectation from one who has seen only rooms and food. Putting names to faces converts an anonymous transaction into a personal one, which is precisely the impression a hotel wishes to create.

5.1.10 Reviews Page :

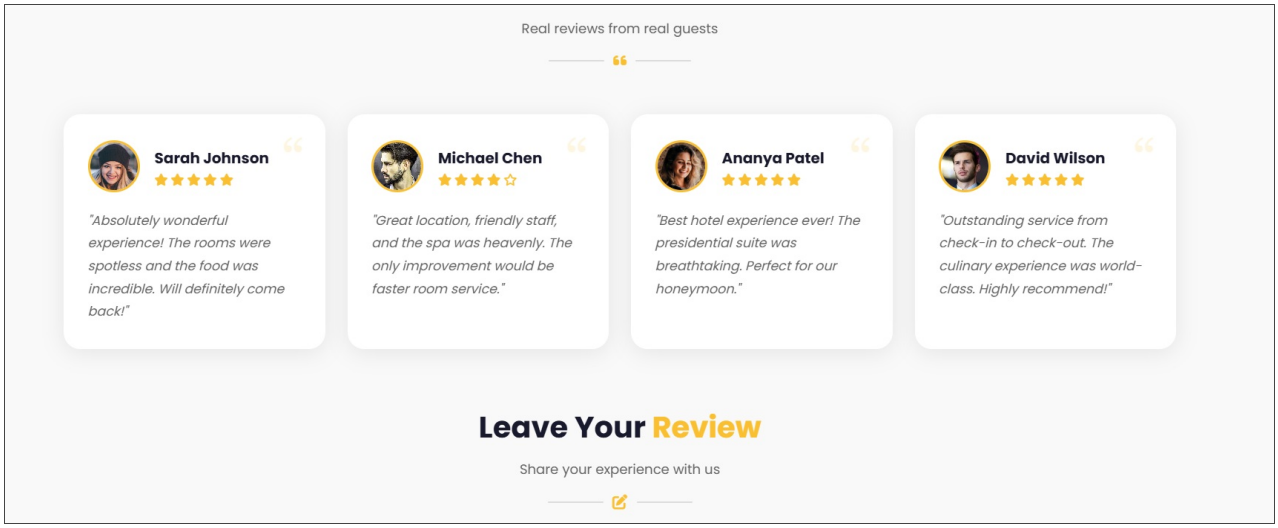


Fig. 5.1.10 — Guest Reviews and Submission Form (review.html)

Description :

The reviews page performs two functions on the same screen: it displays what previous guests

have said, and it collects what the present guest wishes to say. The display section is introduced by the line “Real reviews from real guests” with a quotation-mark divider, and presents testimonials as cards in a four-column grid.

Each review card contains a circular portrait with a gold ring, the reviewer’s name, a star rating rendered as filled and unfilled stars, and the review text in italic within quotation marks. A large decorative quotation mark sits in the upper-right corner of each card. The four visible reviews illustrate the range the system is designed to accommodate: Sarah Johnson gives five stars praising spotless rooms and the food; Michael Chen gives four stars and, while praising the location, staff and spa, notes that room service could be faster; Ananya Patel gives five stars for a honeymoon stay in the presidential suite; and David Wilson gives five stars for service from check-in to check-out.

5.1.11 Order Food Page :

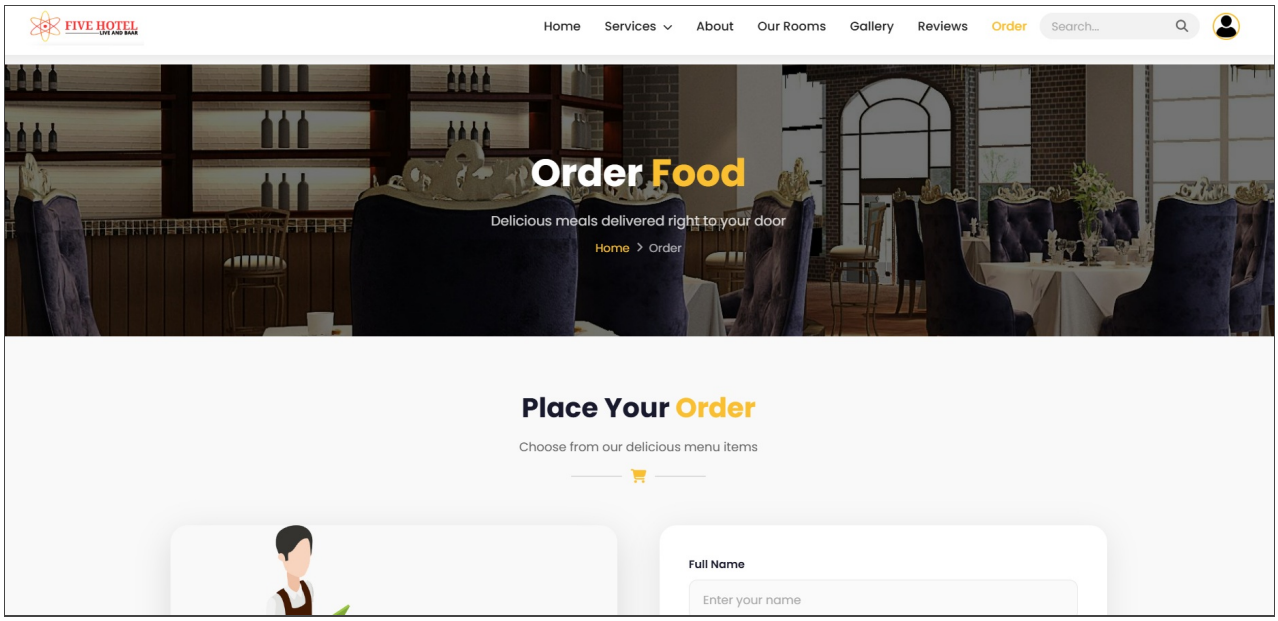


Fig. 5.1.11 — Order Food (order.html)

Description :

The order page is the primary transactional screen of the dining module. Its banner shows the restaurant interior with the heading “Order Food”, the sub-line “Delicious meals delivered right to your door” and the breadcrumb Home › Order. The content area is headed “Place Your Order” with the sub-line “Choose from our delicious menu items” and a cart-icon divider.

The layout is a two-column arrangement. The left column holds an illustration of a waiter, which softens what would otherwise be a bare form and reinforces the service context. The right column holds the order form itself, beginning with the Full Name field visible in the screenshot and continuing with the telephone number, the delivery address, the food selection and the quantity.

5.1.12 Login Page :

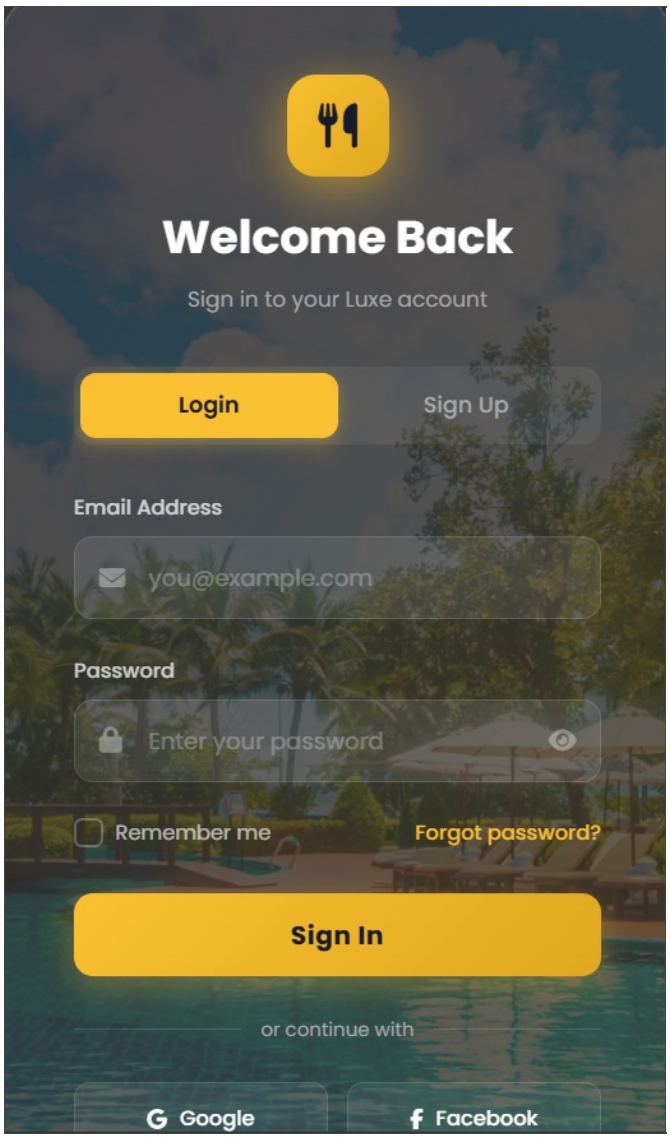


Fig. 5.1.12 — Login (login.html)

Description :

The login screen authenticates a returning user. It is presented as a translucent card floating over a blurred poolside photograph, a treatment that keeps the visual identity of the property present even on a purely functional screen. At the top of the card is a gold app icon, followed by the heading “Welcome Back” and the sub-line “Sign in to your Luxe account”.

Immediately below is a pair of tabs, **Login** and **Sign Up**, with the active tab filled in gold. Placing both authentication paths on a single screen is a considered choice: a visitor who reaches this card without an account can switch to registration in one tap, instead of having to find a separate link and lose their place.

For completeness it must be recorded that the **Forgot password?** link, the **Remember me** option and the two social sign-in buttons are present in the interface but are not backed by an implementation in this release. They are listed among the planned enhancements in Chapter 8.

5.1.13 Sign Up Page :

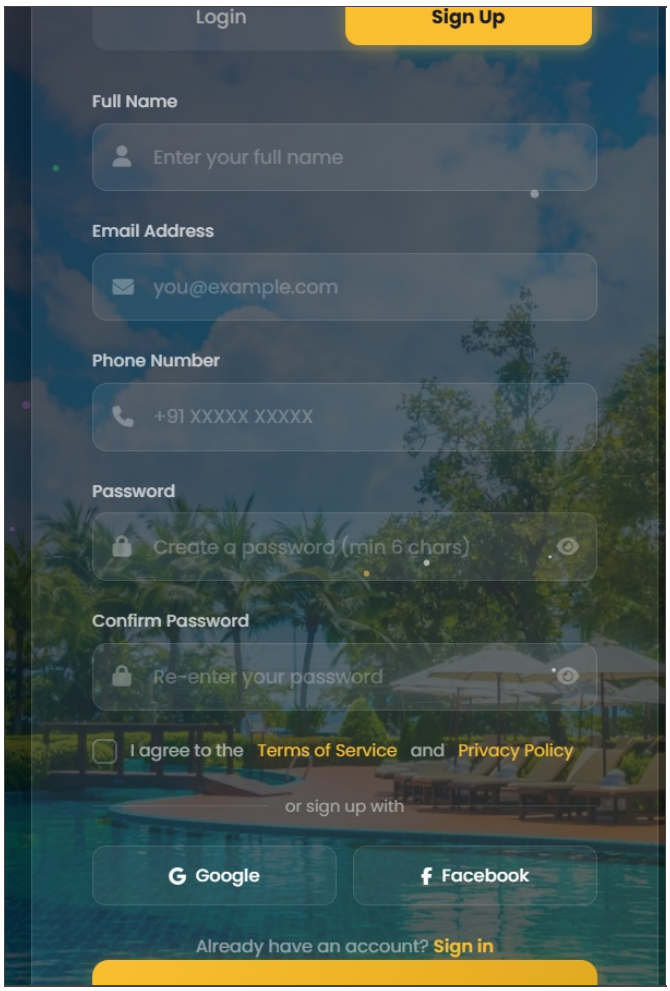


Fig. 5.1.13 — Sign Up (sign-up.html)

Description :

The sign-up screen creates a new account and is reached either directly or by selecting the Sign Up tab on the login card, which is shown filled in gold in the screenshot. The form collects five values in a deliberate order that moves from the least sensitive to the most sensitive.

Full Name carries a person icon and the placeholder “Enter your full name”, and must be at least two characters. **Email Address** carries an envelope icon and the placeholder *you@example.com*, must be a syntactically valid address, and must not already exist in the *users* table. **Phone Number** carries a telephone icon and the placeholder “+91 XXXXX XXXXX”, which communicates both the country context and the expected digit grouping, and must contain at least ten digits.

5.1.14 Payment Page :

Description :

The payment page completes the ordering workflow. It is reached automatically after a successful order submission and collects the information required to bill the customer. The form is divided into two logical groups.

The handling of the card number on this screen is the most security-sensitive operation in the entire application, and it is worth stating exactly what happens. The number is submitted to *payment.php*, where a regular expression strips every non-digit character. The server then takes only the final four digits with *substr()* and binds that four-character value to the *card_last4* column of the prepared INSERT statement.

On success the page displays the message “Payment Successful! Redirecting...” and returns the user to the home page. It must be stated plainly that no real financial settlement occurs: the module records a billing intent and is a simulated checkout, not a live payment facility, as noted in Section 2.3 and Chapter 8.

Summary of Input Validation Rules

Form	Field	Type	Validation Rule
Sign Up	full_name	text	Required; minimum 2 characters
	email	email	Required; valid format; must not already exist in *users*
	phone	tel	Required; minimum 10 digits
	password	password	Required; minimum 6 characters; hashed before storage
	confirm_password	password	Required; must match the password exactly
Login	email	email	Required; valid format; account must exist
	password	password	Required; minimum 6 characters; must verify against the stored hash
Order	name	text	Required; minimum 2 characters
	phone	tel	Required; minimum 10 digits
	address	text	Required; must not be empty
	food_select	select	Required; must be a key present in the server-side price list
	quantity	number	Required; integer; minimum 1
Payment	full_name, city, zip_code	text	All required
	email	email	Required; valid format
	address	text	Required
	card_number	text	Required; reduced to digits only; last 4 retained, remainder discarded
	amount	number	Greater than zero; supplied from the order total, not typed by the user
Review	guest_name	text	Required; minimum 2 characters
	email	email	Required; valid format
	rating	select	Required; integer between 1 and 5
	review_text	textarea	Required; minimum 10 characters

5.2 Output Design / Report Design

Output design covers everything the system produces in response to user action. In this application output takes five distinct forms, each suited to a different situation.

1. Inline Validation Messages

When a field fails a client-side check, a short message appears beside or beneath that field and the field itself is highlighted. This output never involves the server, appears instantly and states the specific problem rather than a general failure.

2. JSON Response Documents

Every back-end endpoint answers with a JSON document rather than with an HTML page, because the request was made asynchronously and the page is expected to update itself rather than reload. The document always contains a *status* key holding either *success* or *error*, and a *message* key holding text suitable for direct display.

```
{ "status": "success",
  "message": "Account created successfully.",
  "user": { "name": "Foram Kapadiya", "email": "foram@example.com" } }

{ "status": "error",
  "message": "This email is already registered. Please log in instead." }
```

3. Success Panels and Toast Notifications

When an action completes, the interface confirms it visibly. Registration replaces the form with a success panel.

4. Navigational Output

Some responses are expressed as movement rather than as text. A successful login returns the user to the home page with the navigation bar now showing their name.

5. Persistent Database Records

The most durable output of the system is the row written into MySQL. Each such row carries an automatically generated primary key, an automatic timestamp and, where applicable, a status value, so that the record is self-describing and can be located, ordered and filtered later without reference to any external index.

Output Message Catalogue

Module	Condition	Message Presented	Presentation
Registration	Success	Account created successfully	Success panel, then redirect
Registration	Duplicate e-mail	This email is already registered	Inline error on the form
Registration	Validation failure	Field-specific message	Inline error beside the field
Login	Success	Login successful	Redirect to index.html
Login	Wrong password	Incorrect password. Please try again.	Inline error on the form
Login	Unknown account	No account found with this email address	Inline error on the form
Order	Success	Order placed successfully, with the computed total	Confirmation, then redirect to payment.html
Order	Invalid item	Please select a valid food item	Inline error on the select control
Payment	Success	Payment Successful! Redirecting...	Confirmation banner, then redirect
Payment	Missing field	Field-specific required message	Inline error beside the field
Review	Success	Review submitted successfully	Toast notification
Review	Text too short	Review must be at least 10 characters	Inline error on the textarea
Review	Rating out of range	Rating must be between 1 and 5	Inline error on the select control
Any endpoint	Wrong HTTP method	Invalid request method	JSON error document
Any endpoint	Connection failure	Database connection failed	JSON error document

Reports Not Implemented

The present version produces no printed or on-screen management report. There is no dashboard listing the day's orders, no revenue summary, no occupancy statement and no exportable file. Operational data must be inspected directly through phpMyAdmin. Reporting is identified in Section 8.3 as one of the most valuable of the planned enhancements, since the necessary data is already being captured in a structured and timestamped form and only the presentation layer is missing.

6- Coding (Few Code Snippets)

This chapter presents representative extracts from the source code of the project. The snippets have been chosen to cover every layer of the system — the connection layer, the back-end controllers, the front-end submission logic and the database schema — rather than to reproduce the code in full.

Snippet 1 — Database Connection

File: php/db_connect.php | Purpose: establish the shared MySQLi connection and set the character encoding.

```
<?php
define('DB_HOST', 'localhost');
define('DB_USER', '[REDACTED]');
define('DB_PASS', '[REDACTED]');
define('DB_NAME', 'food_website');

$conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);

if ($conn->connect_error) {
    die(json_encode([
        'status'  => 'error',
        'message' => 'Database connection failed: ' . $conn->connect_error
    ]));
}

$conn->set_charset('utf8mb4');
?>
```

This is the single point of configuration for the entire back end. Every controller includes this file rather than opening its own connection, which means a change of host, credentials or database name is made once.

Snippet 2 — Input Sanitisation and Validation (Registration)

File: php/register.php | Purpose: clean and verify every submitted value before use.

```
$full_name = trim($conn->real_escape_string($_POST['full_name'] ?? ''));
$email     = trim($conn->real_escape_string($_POST['email']     ?? ''));
$phone     = trim($conn->real_escape_string($_POST['phone']     ?? ''));
$password  = $_POST['password']         ?? '';
$confirm   = $_POST['confirm_password'] ?? '';

$errors = [];

if (strlen($full_name) < 2) {
    $errors[] = 'Please enter your full name.';
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = 'Please enter a valid email address.';
}

if (strlen(preg_replace('/\D/', '', $phone)) < 10) {
    $errors[] = 'Please enter a valid phone number.';
}

if (strlen($password) < 6) {
    $errors[] = 'Password must be at least 6 characters.';
}

if ($password !== $confirm) {
    $errors[] = 'Passwords do not match.';
}
```

Three techniques are combined here. The null-coalescing operator *??* supplies an empty string for any field the browser did not send, which prevents an undefined-index notice.

Snippet 3 — Duplicate E-mail Check

File: php/register.php | Purpose: prevent two accounts sharing the same address.

```
$check = $conn->prepare("SELECT id FROM users WHERE email = ?");
$check->bind_param('s', $email);
$check->execute();
$check->store_result();

if ($check->num_rows > 0) {
    echo json_encode([
        'status'  => 'error',
        'message' => 'This email is already registered. Please log in instead.'
    ]);
    exit;
}
```

The e-mail column carries a UNIQUE constraint at the database level, so a duplicate would be rejected in any case. This explicit check exists so that the user receives a clear, actionable message that tells them what to do next, rather than a raw constraint-violation error.

Snippet 4 — Password Hashing and User Insertion

File: php/register.php | Purpose: store credentials securely.

```
$hashed_password = password_hash($password, PASSWORD_BCRYPT);

$stmt = $conn->prepare(
    "INSERT INTO users (full_name, email, phone, password)
     VALUES (?, ?, ?, ?)"
);
$stmt->bind_param('ssss', $full_name, $email, $phone, $hashed_password);

if ($stmt->execute()) {
    echo json_encode([
        'status'  => 'success',
        'message' => 'Account created successfully.'
    ]);
} else {
    echo json_encode([
        'status'  => 'error',
        'message' => 'Registration failed. Please try again.'
    ]);
}
```

password_hash() with the BCRYPT algorithm generates a random salt for every call, which means two users who choose the same password receive entirely different stored values. The function is also deliberately slow, so that an attacker who obtains the table cannot test candidate passwords at speed.

Snippet 5 — Login Verification and Session Creation

File: php/login.php | Purpose: authenticate a returning user.

```
session_start();

$stmt = $conn->prepare(
    "SELECT id, full_name, email, password FROM users WHERE email = ?"
);
$stmt->bind_param('s', $email);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows === 0) {
    echo json_encode(['status' => 'error',
        'message' => 'No account found with this email address.']);
    exit;
}

$user = $result->fetch_assoc();

if (!password_verify($password, $user['password'])) {
    echo json_encode(['status' => 'error',
        'message' => 'Incorrect password. Please try again.']);
    exit;
}

$_SESSION['user_id']    = $user['id'];
$_SESSION['user_name']  = $user['full_name'];
$_SESSION['user_email'] = $user['email'];

echo json_encode([
    'status'  => 'success',
    'message' => 'Login successful.',
    'user'    => ['name' => $user['full_name'], 'email' => $user['email']]
]);
```

The stored hash is never compared with a plain string. *password_verify()* extracts the salt and cost factor embedded in the stored value, re-hashes the supplied password with those parameters and performs a timing-safe comparison.

Snippet 6 — Server-Side Order Pricing

File: php/order.php | Purpose: calculate the order total without trusting the browser.

```
$food_prices = [
    'pizza'    => 350,
    'pasta'    => 280,
    'sandwich' => 200,
    'cake'     => 450,
    'burger'   => 250,
    'juice'    => 150,
];

if (!array_key_exists($food_item, $food_prices)) {
    echo json_encode(['status' => 'error',
        'message' => 'Please select a valid food item.']);
    exit;
}

$unit_price  = $food_prices[$food_item];
$total_price = $unit_price * $quantity;
```

This is the most important security control in the ordering module. The browser sends only the item key and the quantity; the price is looked up on the server.

Snippet 7 — Card Masking on Payment

File: php/payment.php | Purpose: avoid storing the full card number.

```
$card_num   = preg_replace('/\D/', '', $_POST['card_number'] ?? '');
$card_last4 = substr($card_num, -4);

$stmt = $conn->prepare(
    "INSERT INTO payments
     (full_name, email, address, city, zip_code, card_name,
      card_last4, amount, status)
     VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'success')"
);
$stmt->bind_param('ssssssssd',
    $full_name, $email, $address, $city, $zip_code,
    $card_name, $card_last4, $amount);
$stmt->execute();
```

The regular expression removes every character that is not a digit, which tolerates the spaces and hyphens that users commonly type. *substr()* with a negative offset returns the final four characters.

Snippet 8 — Review Validation

File: php/review.php | Purpose: enforce rating range and minimum review length.

```
$rating      = (int)($_POST['rating'] ?? 5);
$review_text = trim($conn->real_escape_string($_POST['review_text'] ?? ''));

if ($rating < 1 || $rating > 5) {
    $errors[] = 'Rating must be between 1 and 5.';
}

if (strlen($review_text) < 10) {
    $errors[] = 'Review must be at least 10 characters.';
}
```

The explicit cast to integer means that any non-numeric value submitted for the rating becomes zero and is then rejected by the range test, so no unexpected value can reach the TINYINT column. The minimum length rule maintains the usefulness of the review corpus by rejecting empty or single-word submissions.

Snippet 9 — Client-Side Submission (Registration)

File: sign-up.html | Purpose: validate in the browser and post asynchronously.

```
const formData = new FormData();
formData.append('full_name',        name.value.trim());
formData.append('email',            email.value.trim());
formData.append('phone',            phone.value.trim());
formData.append('password',         pwd.value);
formData.append('confirm_password', confirm.value);

fetch('php/register.php', {
    method: 'POST',
    body:   formData
})
.then(res => res.json())
.then(data => {
    if (data.status === 'success') {
        showSuccessPanel(data.message);
    } else {
        showError(data.message);
    }
})
.catch(err => showError('Network error. Please try again.'));
```

The values are trimmed as they are collected, so that leading and trailing whitespace never reaches the server. Note that *FormData* requires no manual content-type header; the browser sets the multipart boundary automatically.

Snippet 10 — Creating the users Table

File: php/database/ setup script | Purpose: define the account table.

```
CREATE TABLE IF NOT EXISTS users (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    full_name  VARCHAR(100) NOT NULL,
    email      VARCHAR(150) NOT NULL UNIQUE,
    phone      VARCHAR(20)  NOT NULL,
    password   VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Two choices are worth noting. The password column is declared as VARCHAR(255) rather than a shorter length, because a BCRYPT hash occupies sixty characters today but a future algorithm may produce a longer value, and the column should not have to be altered when the hashing cost

is raised.

Snippet 11 — Creating the orders Table

File: php/database/ setup script | Purpose: define the order table.

```
CREATE TABLE IF NOT EXISTS orders (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    user_name   VARCHAR(100) NOT NULL,
    phone       VARCHAR(20)  NOT NULL,
    address     VARCHAR(255) NOT NULL,
    food_item   VARCHAR(100) NOT NULL,
    quantity    INT NOT NULL DEFAULT 1,
    total_price DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    status      ENUM('pending','confirmed','delivered','cancelled')
                DEFAULT 'pending',
    ordered_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

The monetary column is DECIMAL(10,2) rather than FLOAT, because binary floating point cannot represent decimal fractions exactly and would introduce rounding errors in currency arithmetic. The ENUM restricts the status to four defined values at the database level, so an invalid state cannot be stored even if application code were to attempt it.

7- Testing (Manual, Test Cases and Test Data)

Testing was carried out manually throughout development and again as a complete pass once all modules were in place. This chapter records the testing approach, the individual test cases and the test data used.

7.1 Manual Testing

Test Environment

Item	Configuration
Operating system	Windows 11
Server stack	XAMPP: Apache, PHP and MySQL running locally
Database	*food_website*, created through the setup scripts
Browsers	Google Chrome (primary), Microsoft Edge and Mozilla Firefox (verification)
Verification tools	phpMyAdmin for database inspection; browser developer tools for console and network tracing
Access URL	http://localhost/ with the project directory placed in the XAMPP web root

Testing Approach

Unit-level manual testing. Each module was tested immediately after it was written, in isolation from the rest of the system. A form was submitted with correct data and the resulting row was located in phpMyAdmin; the same form was then submitted with each invalid variation in turn to confirm that the corresponding validation rule fired.

Integration testing. Once all modules existed, the workflows that cross module boundaries were exercised as a whole. The most important of these is the order-to-payment chain, in which the total computed by *order.php* must arrive intact on the payment screen.

End-to-end journey testing. The complete guest journey was walked through as a new user would experience it: arrive at the home page, browse the rooms, browse the menu, register an account, log in with it, place an order, complete the payment form and submit a review. The database was inspected after each step.

Negative testing. Every input was deliberately supplied with values it should reject: empty strings, malformed addresses, short passwords, mismatched confirmations, short telephone numbers, zero and negative quantities, ratings outside the permitted range, and review text below the minimum length.

Database testing. The setup scripts were executed on a machine where the database did not exist, to confirm that the schema is created correctly from a clean state. Column types, default values and the UNIQUE constraint on the e-mail column were verified through the phpMyAdmin structure view.

Endpoint behaviour testing. Each PHP endpoint was requested with GET instead of POST to confirm that the request-method guard returns a structured error rather than executing.

Responsive testing. Every page was inspected at desktop, tablet and mobile widths using the device-emulation mode of the browser developer tools, checking that grids reflow, that text remains legible and that no element overflows the viewport horizontally.

Cross-browser testing. The site was opened in Chrome, Edge and Firefox to confirm consistent

layout and identical form behaviour.

7.2 Test Cases

The following table records the formal test cases with their module, scenario, precondition, expected behaviour and outcome.

ID	Module	Scenario	Precondition	Expected Behaviour	Status
TC-01	Registration	Valid registration	Database running; e-mail unused	Account created successfully	Pass
TC-02	Registration	Invalid e-mail format	Form open	Validation error shown	Pass
TC-03	Registration	Password shorter than six characters	Form open	Password rule error shown	Pass
TC-04	Registration	Confirmation does not match	Form open	Mismatch error shown	Pass
TC-05	Registration	Duplicate e-mail address	Account already exists	Duplicate error; no insertion	Pass
TC-06	Registration	Phone number shorter than ten digits	Form open	Phone rule error shown	Pass
TC-07	Login	Valid credentials	Account exists	Session created; redirect to home	Pass
TC-08	Login	Incorrect password	Account exists	Access denied with a clear message	Pass
TC-09	Login	Unregistered e-mail	No matching account	“No account found” message	Pass
TC-10	Login	Both fields left blank	Form open	Required-field errors shown	Pass
TC-11	Order	Valid order placed	Database running	Order stored; redirect to payment	Pass
TC-12	Order	No food item selected	Form open	Selection error shown	Pass
TC-13	Order	Quantity set to zero	Form open	Minimum-quantity error shown	Pass
TC-14	Order	Delivery address left empty	Form open	Required-field error shown	Pass
TC-15	Order	Total recalculated on the server	Order submitted	Stored total equals unit price × quantity	Pass
TC-16	Payment	Valid payment submitted	Order exists	Payment stored with status *success*	Pass
TC-17	Payment	Required billing fields blank	Form open	Field errors shown	Pass
TC-18	Payment	Card number with spaces and hyphens	Form open	Non-digits stripped; last four stored	Pass
TC-19	Payment	Full card number must not persist	Payment submitted	Only *card_last4* present in the table	Pass
TC-20	Review	Valid review submitted	Database running	Review stored; toast shown	Pass

TC-21	Review	Review text below ten characters	Form open	Length error shown	Pass
TC-22	Review	Rating outside 1–5	Request submitted	Range error; no insertion	Pass
TC-23	Navigation	All navigation links followed	Site loaded	Every page loads without error	Pass
TC-24	UI	Mobile viewport rendering	Device emulation active	Layout usable; no overflow	Pass
TC-25	Endpoints	GET request to a POST endpoint	Server running	“Invalid request method” returned	Pass
TC-26	Database	Schema created from a clean state	Database absent	Database and five tables created	Pass
TC-27	Database	Connection failure handled	MySQL stopped	Structured JSON error returned	Pass
TC-28	Security	Password stored only as a hash	Account created	Password column contains a BCRYPT hash	Pass
TC-29	Security	Quote characters in a text field	Form open	Value escaped and stored literally	Pass
TC-30	Compatibility	Behaviour across three browsers	Site loaded	Consistent layout and validation	Pass

7.3 Test Data

The following table records the specific values used during testing, showing for each field a value expected to be accepted and a value expected to be rejected.

ID	Module	Field	Valid Test Data	Invalid Test Data	Rule Verified
TD-01	Registration	full_name	Foram Kapadiya	(empty)	Minimum 2 characters
TD-02	Registration	email	foram@example.com	foramexample.com	Valid address format
TD-03	Registration	email	newuser@example.com	foram@example.com (existing)	Uniqueness
TD-04	Registration	phone	9876543210	98765	Minimum 10 digits
TD-05	Registration	password	Five@2026	five	Minimum 6 characters
TD-06	Registration	confirm_password	Five@2026	Five@2025	Must match the password
TD-07	Login	email	foram@example.com	unknown@example.com	Account must exist
TD-08	Login	password	Five@2026	Wrong@123	Hash verification
TD-09	Order	name	Rahul Sharma	R	Minimum 2 characters

TD-10	Order	phone	9998887776	12345	Minimum 10 digits
TD-11	Order	address	14, Kalawad Road, Rajkot	(empty)	Required field
TD-12	Order	food_select	pizza	biryani	Must exist in the server price list
TD-13	Order	quantity	2	0 or −1	Minimum 1
TD-14	Order	total_price	350 × 2 = 700.00	Submitted value of 1.00	Server-side recalculation
TD-15	Payment	full_name	Foram Kapadiya	(empty)	Required field
TD-16	Payment	email	foram@example.com	foram@@example	Valid address format
TD-17	Payment	zip_code	360001	(empty)	Required field
TD-18	Payment	card_number	4111 1111 1111 1111	abcd	Digits extracted; last 4 stored
TD-19	Payment	amount	700.00	0	Must be greater than zero
TD-20	Review	guest_name	Ananya Patel	A	Minimum 2 characters
TD-21	Review	rating	4	7	Range 1 to 5
TD-22	Review	review_text	Excellent stay, the staff were very helpful.	Nice	Minimum 10 characters
TD-23	Security	Any text field	Normal input	O'Brien & Sons	Escaping of quote characters
TD-24	Security	password column	Stored BCRYPT hash	Plain text value	Hashing before storage

Testing Outcome

All thirty formal test cases and all twenty-four manual checks produced the expected behaviour. No defect remained open at the conclusion of testing. Three limitations of the testing itself should be recorded honestly.

8- Enhancements

8.1 Advantages of Your Project

Integration of an entire guest journey. The clearest advantage of the project is that it brings browsing, identification, ordering, payment and feedback into one continuous experience. A guest arrives knowing nothing about the property and leaves having transacted with it, without at any point being sent to another channel.

Genuine database-backed operation. Every transactional action produces a durable row in MySQL with a typed column set, an automatic timestamp and a status value. The site is not a mock-up with disabled buttons; submitting an order actually creates an order.

Security applied at the right points. Rather than claiming general security, the project applies specific controls where they matter. Passwords are hashed with BCRYPT, so the stored column is useless to anyone who obtains it.

Defence in depth through two-layer validation. Client-side validation gives the user immediate, specific feedback and avoids needless network traffic; server-side validation guarantees integrity even when the browser layer is bypassed entirely. Neither layer is trusted to do the other's work, which is the correct architecture and is frequently omitted in projects of this scale.

Modern asynchronous interaction. All five transactional modules submit through the Fetch API and receive JSON, so the interface updates in place rather than reloading. The result is an application that feels contemporary, and it demonstrates an understanding of how a modern web front end actually communicates with a server.

Professional visual quality. The gold-on-navy identity, the consistent card system, the full-width hero banners, the scroll-reveal animations and the animated statistic counters produce a site that is commercially plausible. In hospitality this is not decoration; presentation is itself part of the product, and a guest who finds the site attractive is more likely to trust the property behind it.

Honest and consistent pricing. The prices shown on the menu cards match the server-side price list exactly, and the total carried into the checkout is the one the server computed. The user therefore never encounters an unexplained difference between the displayed price and the amount billed.

Low cost and effortless deployment. The project requires no licence, no framework, no build step and no dependency installation. It runs on any machine with XAMPP, which makes it equally suitable for academic demonstration and for a small business with a minimal budget.

Readability and maintainability. One file per responsibility, a single shared connection, consistent naming from the form field through the PHP variable to the database column, and no hidden framework behaviour mean that a developer encountering the project for the first time can follow any workflow end to end within a few minutes.

Designed for extension. The *bookings* table is fully specified in the schema, the status columns already support a lifecycle, and the modules are independent of one another. The next stage of development can therefore add capability without restructuring what already exists.

Demonstrated academic competence. Taken together, the project shows interface design, responsive layout, client-side scripting, asynchronous communication, server-side programming, input validation, relational schema design, database connectivity, session management and applied security — that is, the full range of skills a full-stack project is intended to evidence.

8.2 Limitation of Your Project

The following limitations are stated plainly. Each is a real boundary of the present release rather than a defect, and each corresponds to an item in the future scope.

1. No administrative interface. The most significant limitation.

2. The booking workflow is not connected. The *bookings* table is fully designed, and the room cards carry a **Book Now** button, but no form writes a reservation and no availability logic exists. Accommodation, which is the primary revenue stream of a hotel, is therefore presented but not transactable in this release.

3. Payment is simulated. No payment gateway is integrated. The module records billing intent and marks the row as successful, but no money moves, no authorisation occurs and no receipt is issued. The system is not PCI-DSS compliant and must not be used for real transactions.

4. No role-based authorisation. The session stores the user's identity but not any role or permission.

5. No route protection. No page checks whether a session exists before rendering.

6. No logout facility. The session is created at login but the application provides no way to destroy it.

7. Search is presentational. The search field in the navigation bar and the quick-search panel on the home page accept input but have no back end. Nothing is queried and no results are produced.

8. Incomplete authentication features. The **Forgot password?** link, the **Remember me** checkbox and the Google and Facebook sign-in buttons appear in the interface but are not implemented. A user who forgets a password currently has no recovery path at all.

9. Content is hard-coded. Room details, tariffs, menu items and prices are written directly into the HTML and into the PHP price array.

10. No automated e-mail or SMS. The system issues no order confirmation, no payment receipt and no notification of any kind.

11. No reporting or analytics. No dashboard, chart, summary or export exists.

12. No update or delete operations. The application only creates records.

13. No automated testing. All verification was manual.

14. Local deployment only. The project is configured for XAMPP on a development machine.

15. Inconsistent database naming. Some setup scripts refer to a database named *luxe_hotel*, while the runtime connection in *db_connect.php* uses *food_website*.

16. No referential integrity. No foreign key links a transaction to a user account.

17. Missing web security controls. There is no CSRF token on any form, no rate limiting on the login endpoint to slow brute-force attempts, no output escaping when stored review text is redisplayed, and no audit logging.

8.3 Future Scope

The enhancements below are ordered by priority, beginning with those that remove the most serious present limitations.

Phase 1 — Completing the Core Application

Administrative dashboard. The highest priority. A protected area listing orders, payments,

reviews and users, with filtering by date and status, the ability to advance an order through its lifecycle, and the ability to remove an inappropriate review.

Role-based authorisation and route protection. A *role* column on the *users* table, a session check at the head of every protected page, and a redirect for unauthorised access. This is the prerequisite for the dashboard and closes limitations 4 and 5 together.

Logout. A short script calling *session_unset()* and *session_destroy()*, with a link in the navigation bar shown only when a session exists.

Room booking workflow. Connecting the **Book Now** button to a reservation form that writes into the already-defined *bookings* table, with date validation ensuring that check-out follows check-in, a tariff calculation based on the number of nights, and an availability check against existing confirmed bookings for the same room.

Password recovery. A token-based reset flow: the user requests a reset, a single-use token with a short expiry is generated and e-mailed, and the token is exchanged for the right to set a new password.

Phase 2 — Operational Maturity

Content management. Moving rooms, menu items and prices from hard-coded HTML into database tables, with administrative screens for editing them. This removes limitation 9 entirely and guarantees that the displayed price and the charged price can never diverge, because both would read the same row.

Automated notifications. E-mail confirmation on registration, order and payment, and optional SMS for delivery updates, implemented with PHPMailer against an SMTP account.

Order lifecycle tracking. Exposing the existing status column to the customer as a progress indicator, so that an order can be followed from pending through confirmed to delivered.

Update and cancel operations. Allowing a customer to cancel an order within a defined window and to correct a delivery address, and allowing an administrator to amend records on their behalf.

Working search. A back end that queries rooms and menu items by keyword, price band and category, returning results through the same asynchronous pattern used elsewhere.

Phase 3 — Security and Production Readiness

CSRF protection. A per-session token embedded in every form and verified on the server before any state-changing operation.

Rate limiting and account lockout. Counting failed login attempts per address and per IP, and applying an increasing delay or a temporary lock, so that the login endpoint cannot be brute-forced.

Output escaping. Passing all stored user text through *htmlspecialchars()* before it is redisplayed, which closes the stored cross-site-scripting vector in the review module.

Transport security and secure cookies. Serving the site over HTTPS and marking the session cookie *HttpOnly*, *Secure* and *SameSite*.

Audit logging. Recording administrative actions with the actor, the action and a timestamp, so that any change to a record can be traced.

Referential integrity. Adding a *user_id* foreign key to the transactional tables once authentication gates those actions, which enables a genuine per-customer history.

Real payment gateway. Integration with a provider such as Razorpay, PayU or Stripe, so that

authorisation, capture, refund and receipt generation are handled by a compliant third party and no card data touches the application at all.

Phase 4 — Growth Features

Customer dashboard. A personal area showing order history, past bookings, submitted reviews and saved addresses.

Reporting and analytics. Daily and monthly revenue summaries, best-selling menu items, occupancy by room type, average rating over time, and exportable CSV or PDF statements. All the underlying data is already being captured with timestamps, so only the presentation layer is missing.

Loyalty programme. Points accrued per transaction and redeemable against future orders, built on the customer identity established in Phase 1.

Multilingual interface. Support for Hindi and Gujarati in addition to English, which would substantially widen the accessible audience for a property in Gujarat.

Progressive web application. A service worker and manifest so that the site can be installed on a phone and can display cached content when the connection is poor.

Automated test suite. PHPUnit tests for the validation and pricing logic and browser-driven tests for the critical journeys, executed before every release.

Cloud deployment. Migration to a managed host with a custom domain, automated backups and monitoring, which is the final step from academic project to live service.

Concluding Remark

The project as delivered establishes a sound foundation: the data model is correct, the security decisions taken are the right ones, the architecture is clean and the guest journey is complete from browsing to payment. What remains is largely additive rather than corrective.

9- References

#	Technology / Source	Reference	Purpose in the Project
1	HTML5	MDN Web Docs — HTML. Mozilla Foundation. https://developer.mozilla.org/en-US/docs/Web/HTML	Page structure, semantic markup and form elements
2	CSS3	MDN Web Docs — CSS. Mozilla Foundation. https://developer.mozilla.org/en-US/docs/Web/CSS	Layout, grid systems, responsive rules and animation
3	JavaScript	MDN Web Docs — JavaScript. Mozilla Foundation. https://developer.mozilla.org/en-US/docs/Web/JavaScript	Client-side validation and interface behaviour
4	Fetch API	MDN Web Docs — Using the Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API	Asynchronous form submission and JSON handling
5	PHP	PHP Manual. The PHP Group. https://www.php.net/docs.php	Server-side processing and request handling
6	PHP Password Hashing	PHP Manual — Password Hashing Functions. https://www.php.net/manual/en/book.password.php	BCRYPT hashing and verification of credentials
7	MySQLi	PHP Manual — MySQLi. https://www.php.net/manual/en/book.mysqli.php	Database connectivity and prepared statements
8	PHP Sessions	PHP Manual — Session Handling. https://www.php.net/manual/en/book.session.php	Retention of the authenticated user identity
9	MySQL	MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/	Schema definition, data types and constraints
10	XAMPP	Apache Friends — XAMPP Documentation. https://www.apachefriends.org/	Local Apache, PHP and MySQL environment
11	phpMyAdmin	phpMyAdmin Documentation. https://docs.phpmyadmin.net/	Database inspection and verification during testing
12	Font Awesome	Font Awesome Documentation, version 6.5.0. https://fontawesome.com/docs	Interface icons throughout the site
13	Intersection Observer	MDN Web Docs — Intersection Observer API. https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API	Scroll-reveal animation of page sections
14	Web Storage API	MDN Web Docs — Window.localStorage. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage	Retention of the display name and order total between pages
15	OWASP	OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/	Reference for the security controls applied and the gaps identified in Section 8.2
16	W3Schools	W3Schools Online Web Tutorials — HTML, CSS, JavaScript, PHP, SQL. https://www.w3schools.com/	Syntax reference during implementation